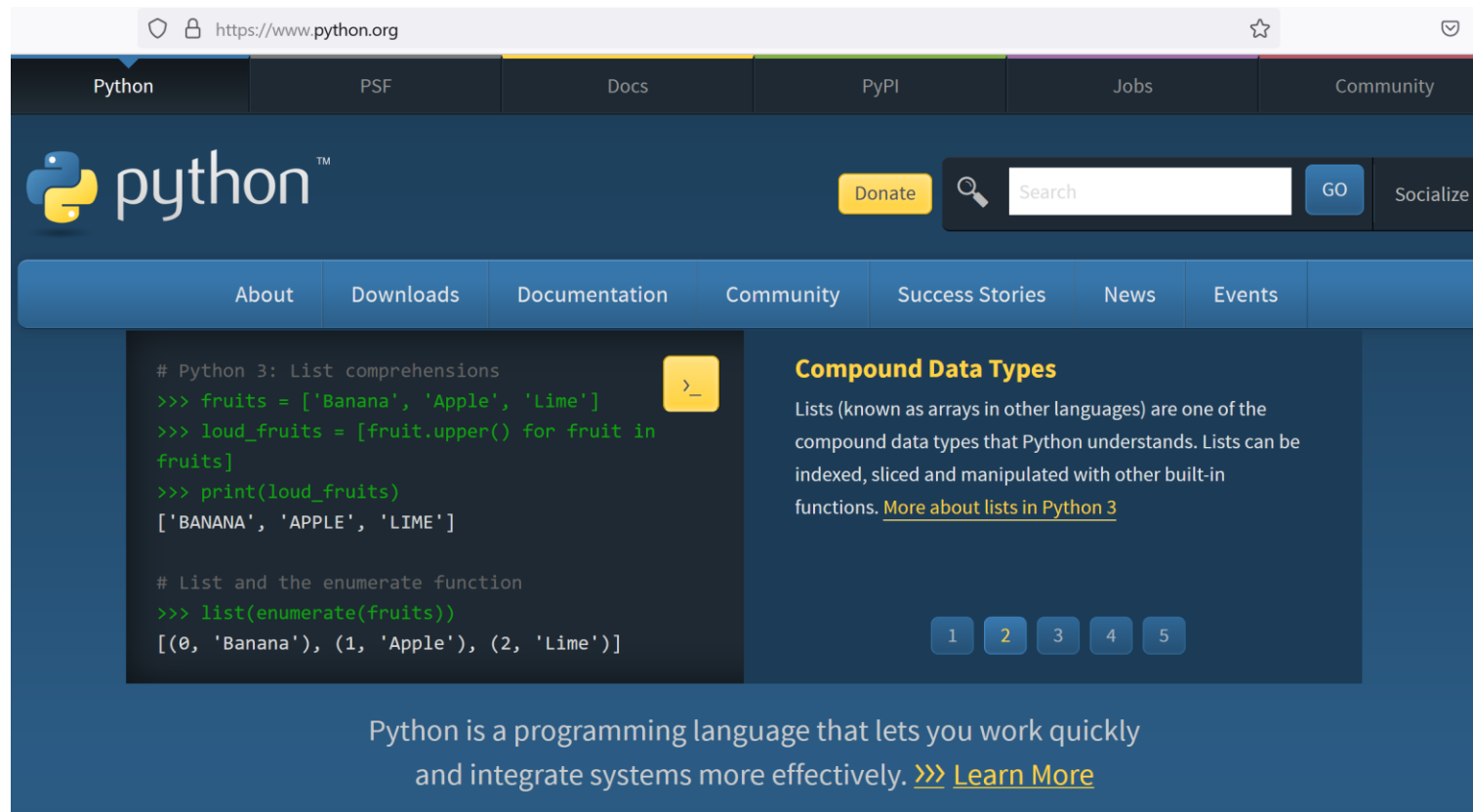


Install Python

- <https://www.python.org/>



Install Jupyter

```
python --version
```

```
pip install jupyter
```

```
jupyter notebook
```



File View Settings Help

Files Running

Select items to perform actions on them.

/



Untitled1.ipynb ☆

File Edit View Insert Runtime Tools Help [All changes saved](#)



+ Code + Text



Google Colab

- Colab is a free Jupyter notebook environment that runs entirely in the cloud
- it does not require a setup and the notebooks that you create can be simultaneously edited by your team members
- Colab supports many popular machine learning libraries which can be easily loaded in your notebook

What Colab Offers You?

As a programmer, you can perform the following using Google Colab.

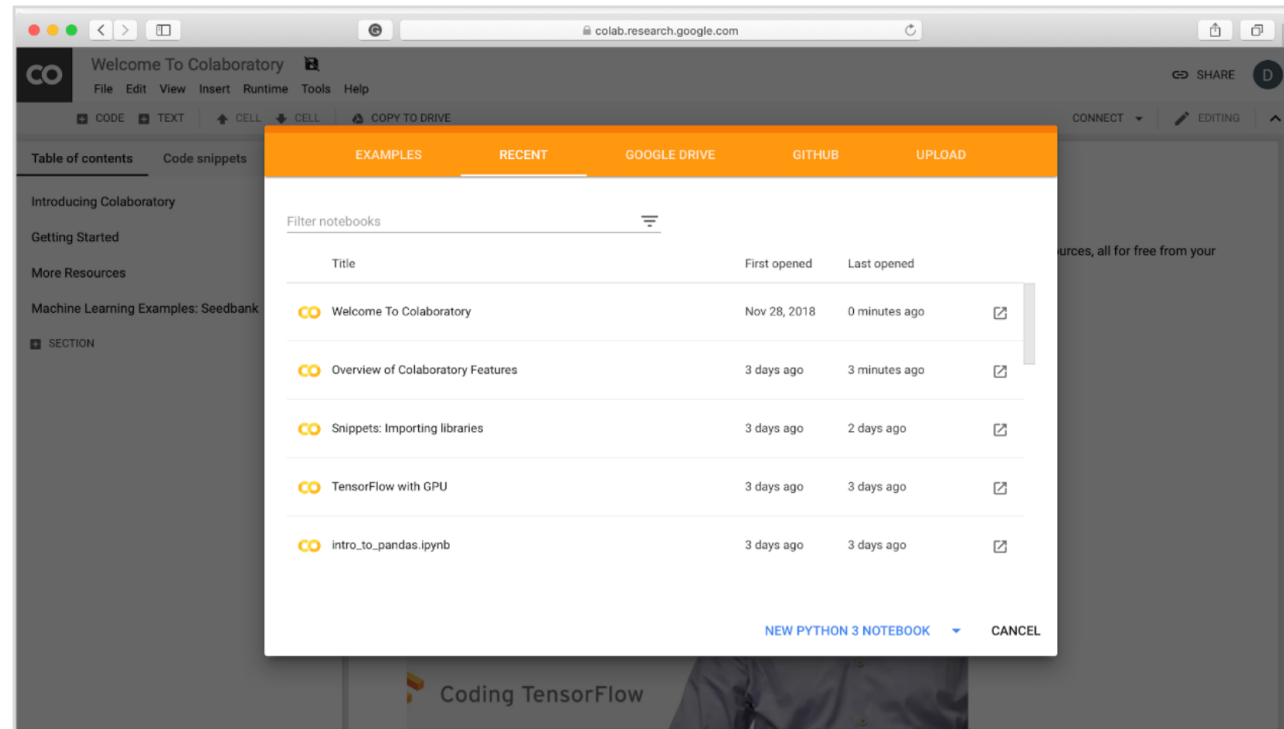
- Write and execute code in Python
- Document your code that supports mathematical equations
- Create/Upload/Share notebooks
- Import/Save notebooks from/to Google Drive
- Import/Publish notebooks from GitHub
- Import external datasets e.g. from Kaggle
- Integrate PyTorch, TensorFlow, Keras, OpenCV
- Free Cloud service with free GPU

Google Colab – Your First Colab Notebook

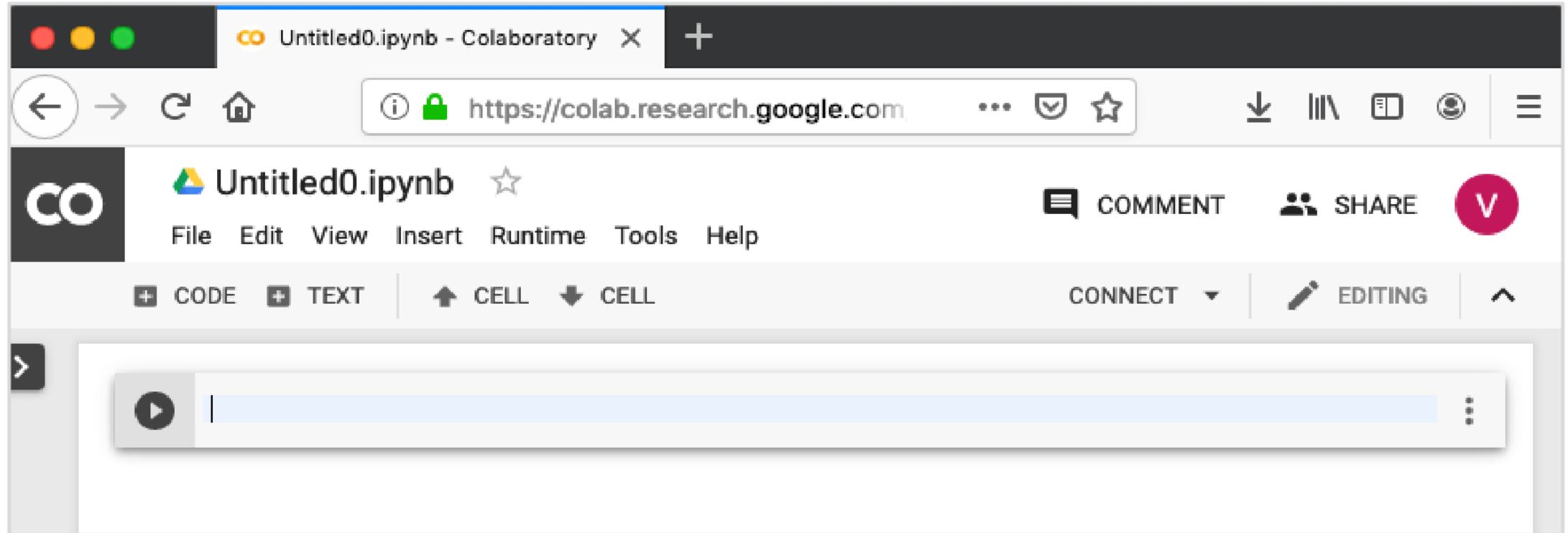
- log in to your Google Drive account before proceeding further

Step 1: Open the following URL in your browser: <https://colab.research.google.com>

Your browser would display the following screen (assuming that you are logged into your Google Drive):



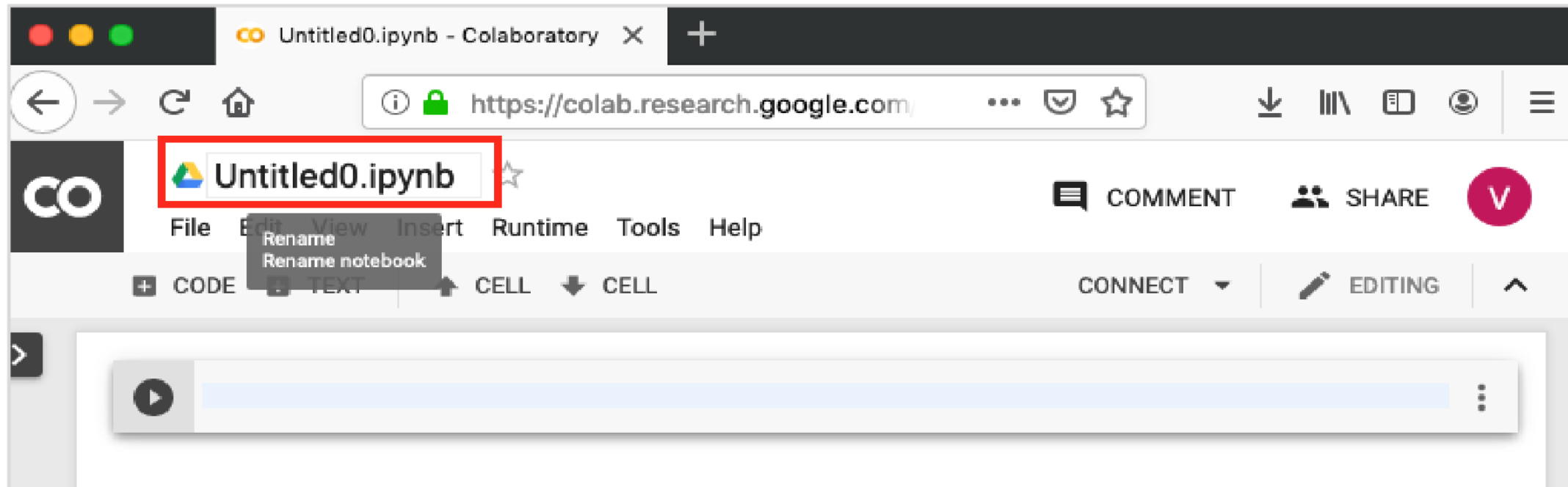
Step 2: Click on the **NEW PYTHON 3 NOTEBOOK** link at the bottom of the screen. A new notebook would open up as shown in the screen below.



As you might have noticed, the notebook interface is quite similar to the one provided in Jupyter. There is a code window in which you would enter your Python code.

Setting Notebook Name

By default, the notebook uses the naming convention **UntitledXX.ipynb**. To rename the notebook, click on this name and type in the desired name in the edit box as shown here:



We will call this notebook as **MyFirstColabNotebook**. So type in this name in the edit box and hit ENTER. The notebook will acquire the name that you have given now.

Entering Code

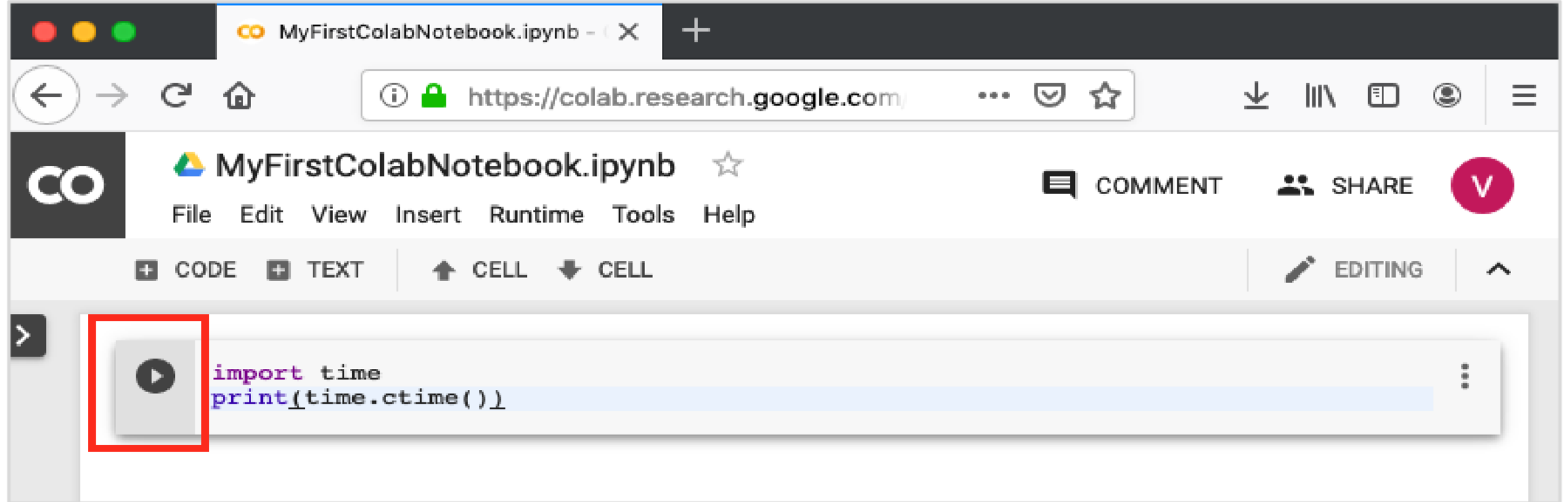
You will now enter a trivial Python code in the code window and execute it.

Enter the following two Python statements in the code window:

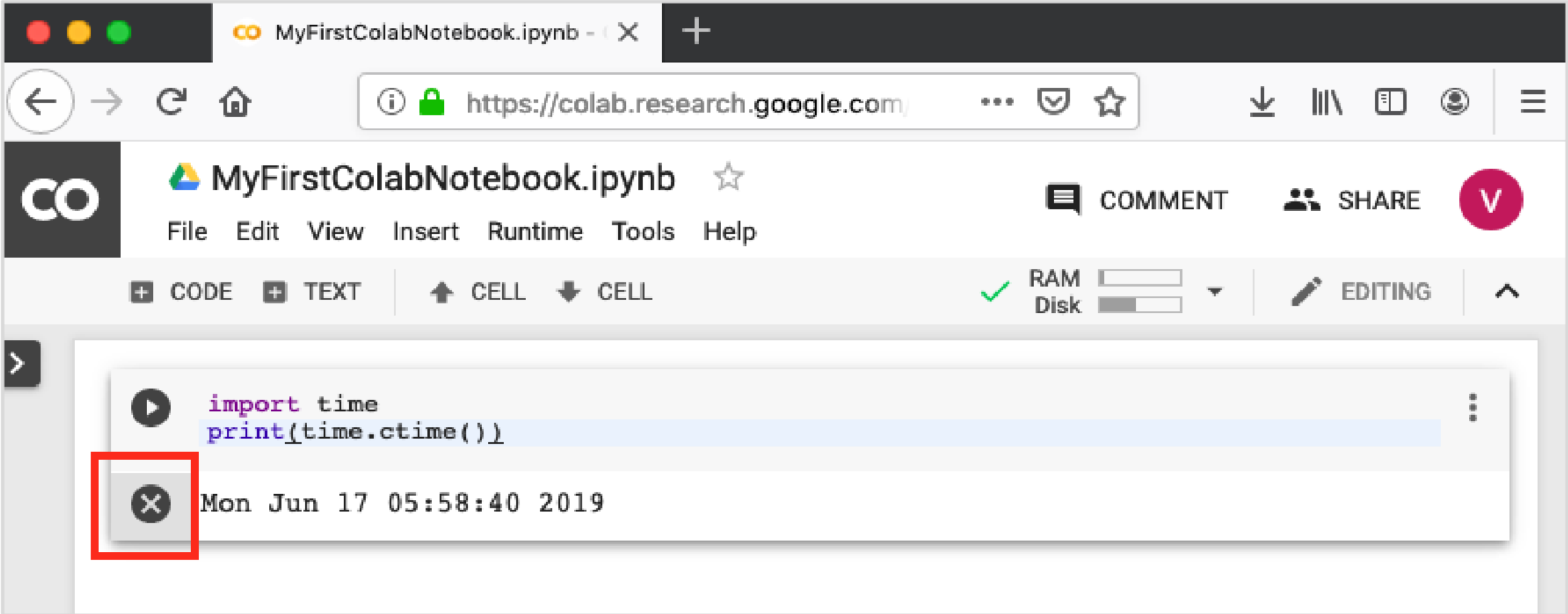
```
import time  
print(time.ctime())
```

Executing Code

To execute the code, click on the arrow on the left side of the code window.



You can clear the output anytime by clicking the icon on the left side of the output display.

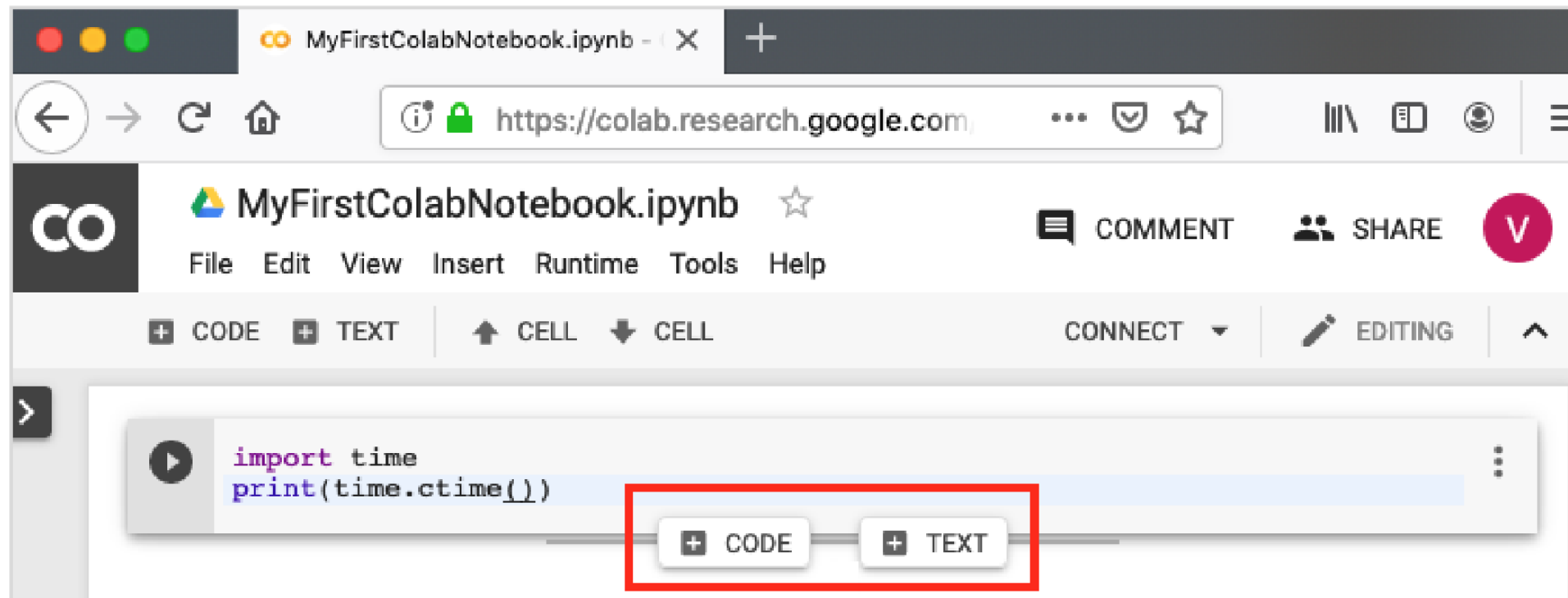


Adding Code Cells

To add more code to your notebook, select the following **menu** options:

Insert / Code Cell

Alternatively, just hover the mouse at the bottom center of the Code cell. When the **CODE** and **TEXT** buttons appear, click on the CODE to add a new cell. This is shown in the screenshot below:



A new code cell will be added underneath the current cell. Add the following two statements in the newly created code window:

```
time.sleep(5)  
print (time.ctime())
```

Now, if you run this cell, you will see the following output:

```
Mon Jun 17 04:50:27 2019
```

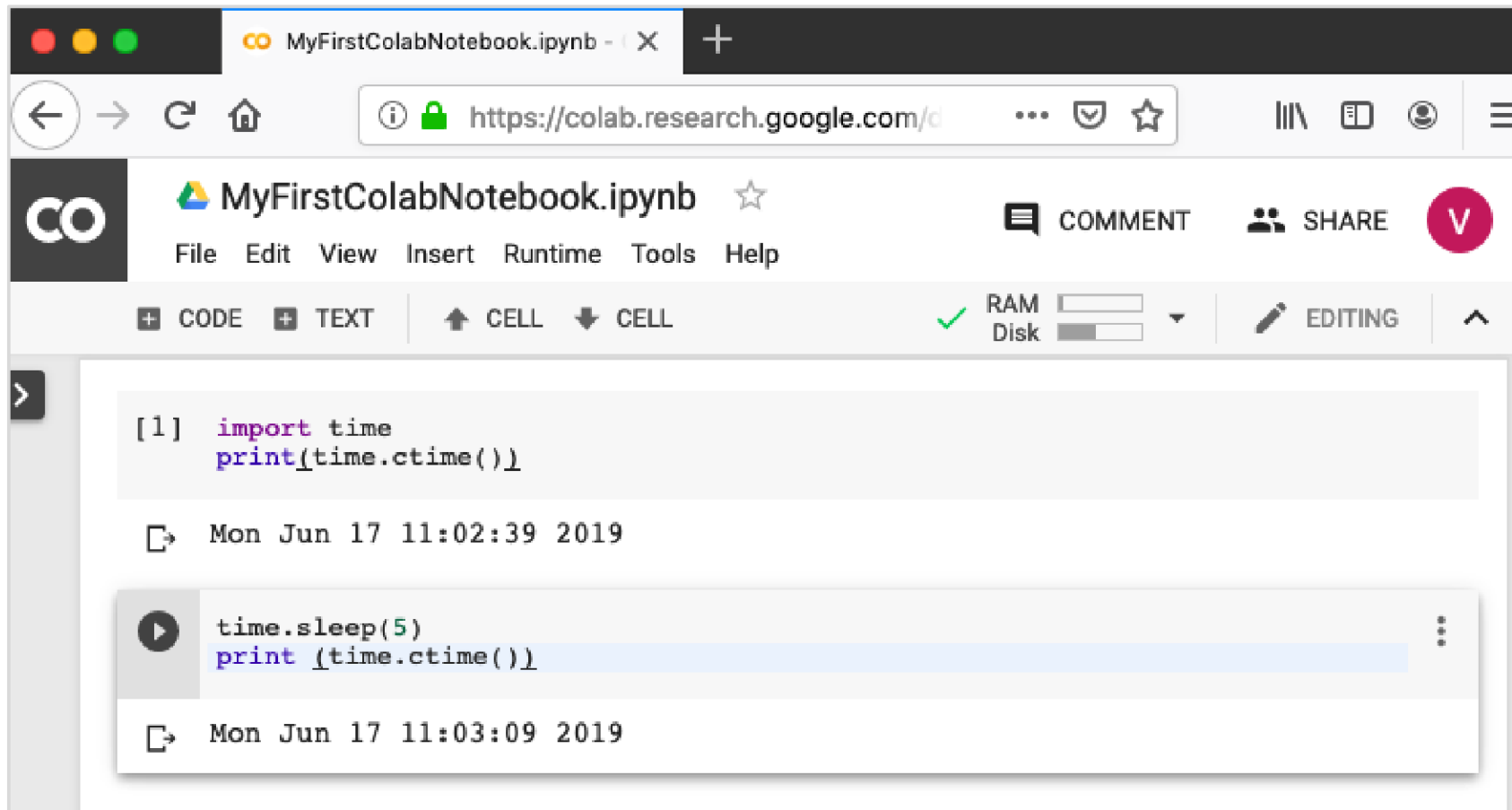
Certainly, the time difference between the two time strings is not 5 seconds. This is obvious as you did take some time to insert the new code. Colab allows you to run all code inside your notebook without an interruption.

Run All

To run the entire code in your notebook without an interruption, execute the following menu options:

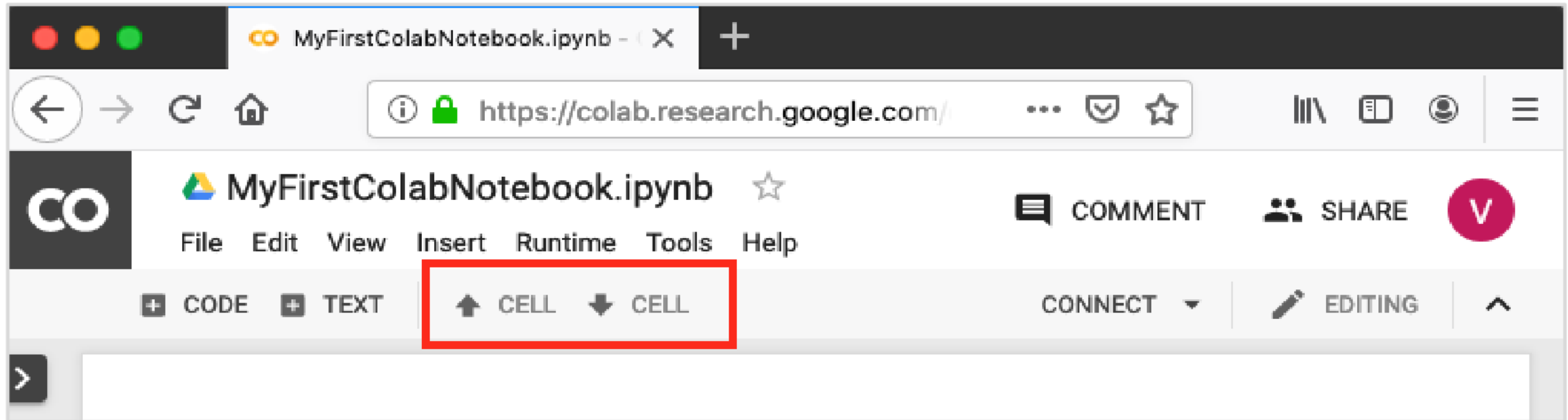
Runtime / Reset and run all...

It will give you the output as shown below:



Changing Cell Order

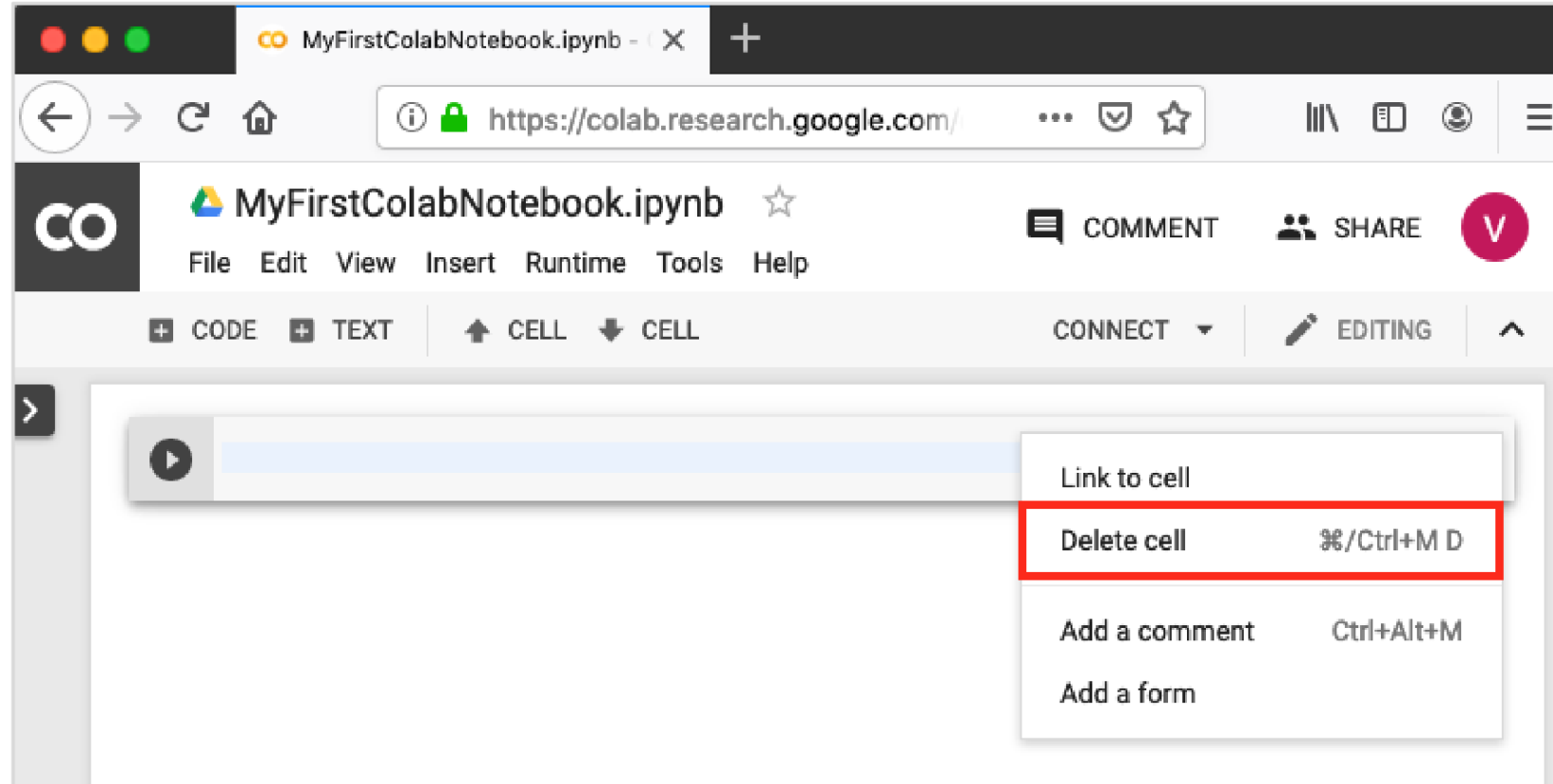
When your notebook contains a large number of code cells, you may come across situations where you would like to change the order of execution of these cells. You can do so by selecting the cell that you want to move and clicking the **UP CELL** or **DOWN CELL** buttons shown in the following screenshot:



You may click the buttons multiple times to move the cell for more than a single position.

Deleting Cell

During the development of your project, you may have introduced a few now-unwanted cells in your notebook. You can remove such cells from your project easily with a single click. Click on the vertical-dotted icon at the top right corner of your code cell.



Click on the **Delete cell** option and the current cell will be deleted.

Now, as you have learned how to run a trivial notebook, let us explore the other capabilities of Colab.

Documenting Your Code

- you may use Python **comments in the code** window to describe your code
- ML heavily uses mathematics and to explain those terms and equations to your readers you need an editor that supports LaTeX - a language for mathematical representations
- Colab provides **Text Cells for this purpose.**

Markdown Examples

Let us look into few examples of markup language syntax to demonstrate its capabilities.

Type in the following text in the Text cell.

```
This is **bold**.
```

```
This is *italic*.
```

```
This is ~strikethrough~.
```

The output of the above commands is rendered on the right hand side of the **Cell** as shown here.

Mathematical Equations

Add a **Text Cell** to your notebook and enter the following markdown syntax in the text window:

```
\sqrt{3x-1}+(1+x)^2
```

You will see the immediate rendering of the markdown code in the right hand side panel of the text cell. This is shown in the screenshot below:



Hit **Enter** and the markdown code disappears from the text cell and only the rendered output is shown.

Let us try another more complicated equation as shown here:

```
e^x=\sum_{i=0}^{\infty} \frac{1}{i!}x^i
```

The rendered output is shown here for your quick reference.

$$e^x = \sum_{i=0}^{\infty} \frac{1}{i!} x^i$$

Code for Sample Equations

Here is the code for the sample equations shown in an earlier screenshot:

Constraints are

- $3x_1 + 6x_2 + x_3 \leq 28$
- $7x_1 + 3x_2 + 2x_3 \leq 37$
- $4x_1 + 5x_2 + 2x_3 \leq 19$
- $x_1, x_2, x_3 \geq 0$

The trial vector is calculated as follows:

$$u_i(t) = x_i(t) + \beta(\hat{x}(t) - x_i(t)) + \beta \sum_{k=1}^n \alpha_v(x_{i1,k}(t) - x_{i2,k}(t))$$

$$f(x_1, x_2) = 20 + e^{-20 \exp(-0.2 \sqrt{\frac{1}{n} (x_1^2 + x_2^2)}) - \exp(\frac{1}{n} (\cos(2\pi x_1) + \cos(2\pi x_2)))}$$

$$x \in [-5, 5]$$

$$>A_{\{m,n\}} =$$

`\begin{pmatrix}`

`a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\`

`a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\`

`\vdots & \vdots & \ddots & \vdots \\`

`a_{m,1} & a_{m,2} & \cdots & a_{m,n}`

`\end{pmatrix}`

Constraints are

- $3x_1 + 6x_2 + x_3 \leq 28$
- $7x_1 + 3x_2 + 2x_3 \leq 37$
- $4x_1 + 5x_2 + 2x_3 \leq 19$
- $x_1, x_2, x_3 \geq 0$

The trial vector is calculated as follows:

- $u_i(t) = x_i(t) + \beta(\hat{x}(t) - x_i(t)) + \beta \sum_{k=1}^{n_v} (x_{i1,k}(t) - x_{i2,k}(t))$

$$f(x_1, x_2) = 20 + e - 20 \exp(-0.2 \sqrt{\frac{1}{n}(x_1^2 + x_2^2)}) - \exp(\frac{1}{n}(\cos(2\pi x_1) + \cos(2\pi x_2))) \quad x \in [-5, 5]$$

$$A_{m,n} = \begin{pmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{pmatrix}$$

Saving Your Work

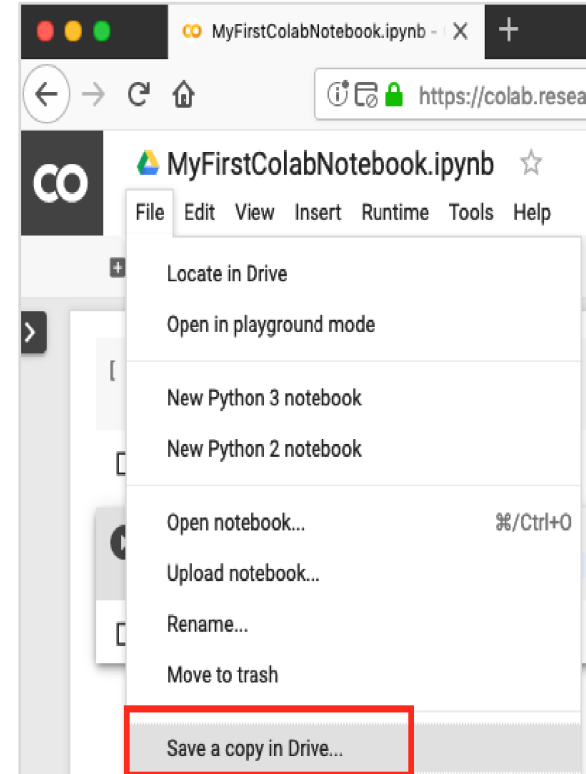
Colab allows you to save your work to Google Drive or even directly to your GitHub repository.

Saving to Google Drive

Colab allows you to save your work to your Google Drive. To save your notebook, select the following menu options:

File / Save a copy in Drive...

You will see the following screen:

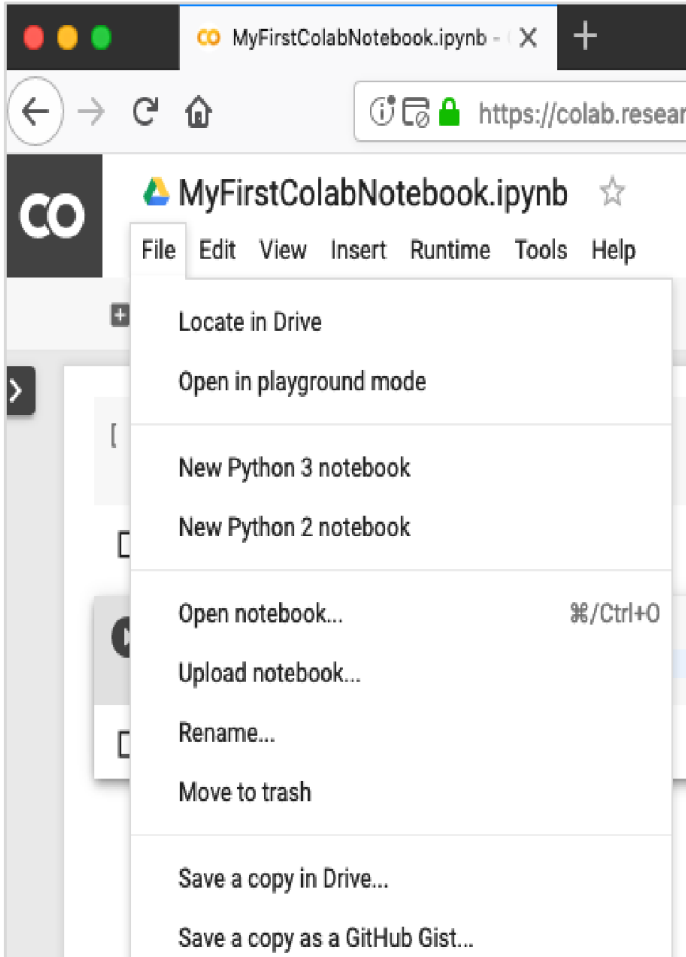


Saving to GitHub

You may also save your work to your GitHub repository by selecting the following menu options:

```
File / Save a copy in GitHub...
```

The menu selection is shown in the following screenshot for your quick reference:







Sign in to **GitHub**
to continue to **Colaboratory**

Username or email address

Password

[Forgot password?](#)

Sign in

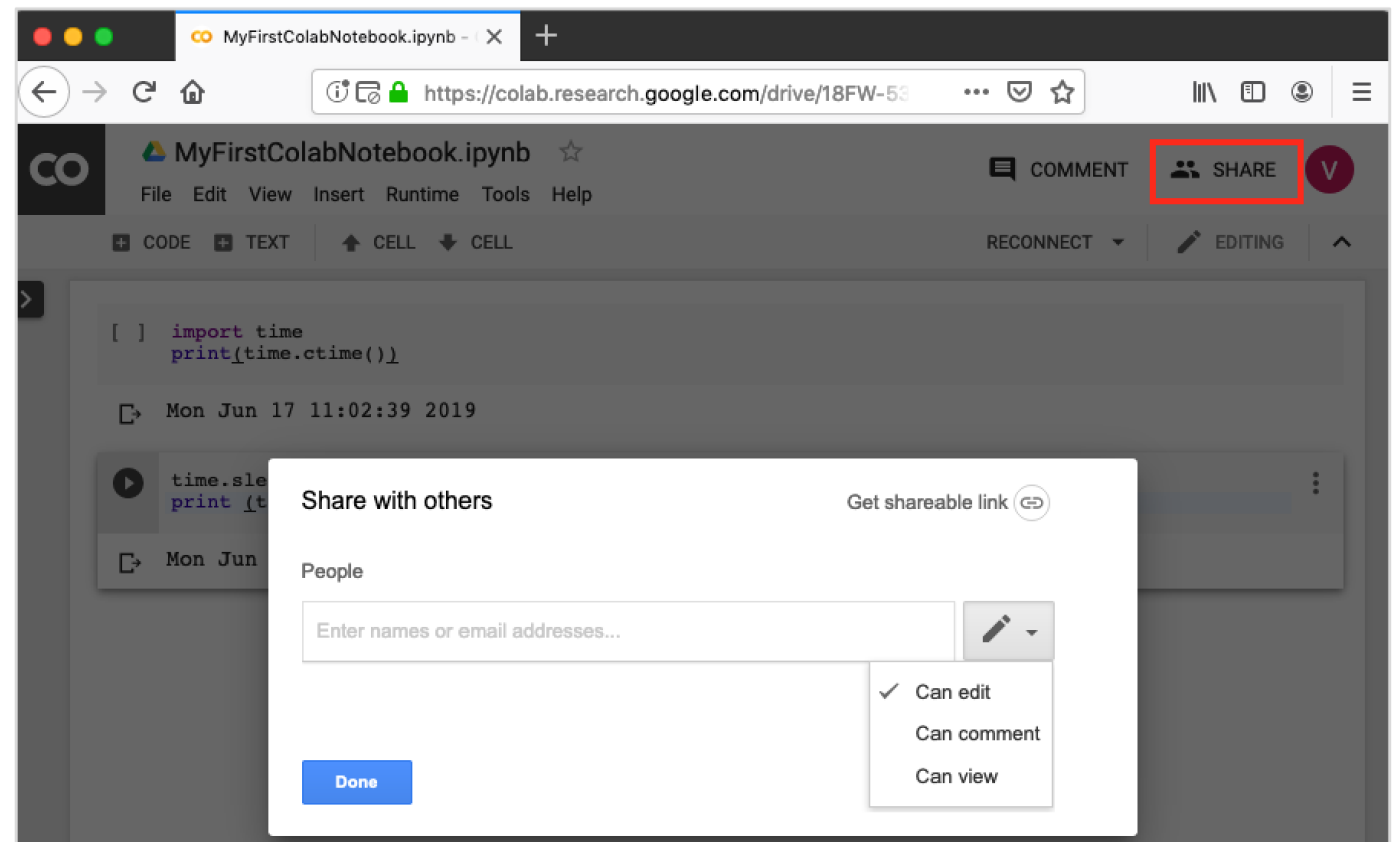
New to GitHub? [Create an account.](#)

Sharing Notebook

To share the notebook that you have created with other co-developers, you may share the copy that you have made in your Google Drive.

To publish the notebook to general audience, you may share it from your GitHub repository.

There is one more way to share your work and that is by clicking on the **SHARE** link at the top right hand corner of your Colab notebook. This will open the share box as shown here:



You may enter the email IDs of people with whom you would like to share the current document. You can set the kind of access by selecting from the three options shown in the above screen.

Click on the **Get shareable link** option to get the URL of your notebook. You will find options for whom to share as follows:

- Specified group of people
- Colleagues in your organization
- Anyone with the link
- All public on the web

Invoking System Commands

Getting Remote Data

Let us look into another example that loads the dataset from a remote server. Type in the following command in your Code cell:

```
!wget http://mlr.cs.umass.edu/ml/machine-learning-databases/adult/adult.data -P  
"/content/drive/My Drive/app"
```

If you run the code, you would see the following output:

```
--2019-06-20 10:09:53--  http://mlr.cs.umass.edu/ml/machine-learning-  
databases/adult/adult.data  
Resolving mlr.cs.umass.edu (mlr.cs.umass.edu)... 128.119.246.96  
Connecting to mlr.cs.umass.edu (mlr.cs.umass.edu)|128.119.246.96|:80...  
connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 3974305 (3.8M) [text/plain]  
Saving to: '/content/drive/My Drive/app/adult.data.1'  
  
adult.data.1      100%[=====>]    3.79M  1.74MB/s   in 2.2s  
  
2019-06-20 10:09:56 (1.74 MB/s) - '/content/drive/My Drive/app/adult.data.1'  
saved [3974305/3974305]
```

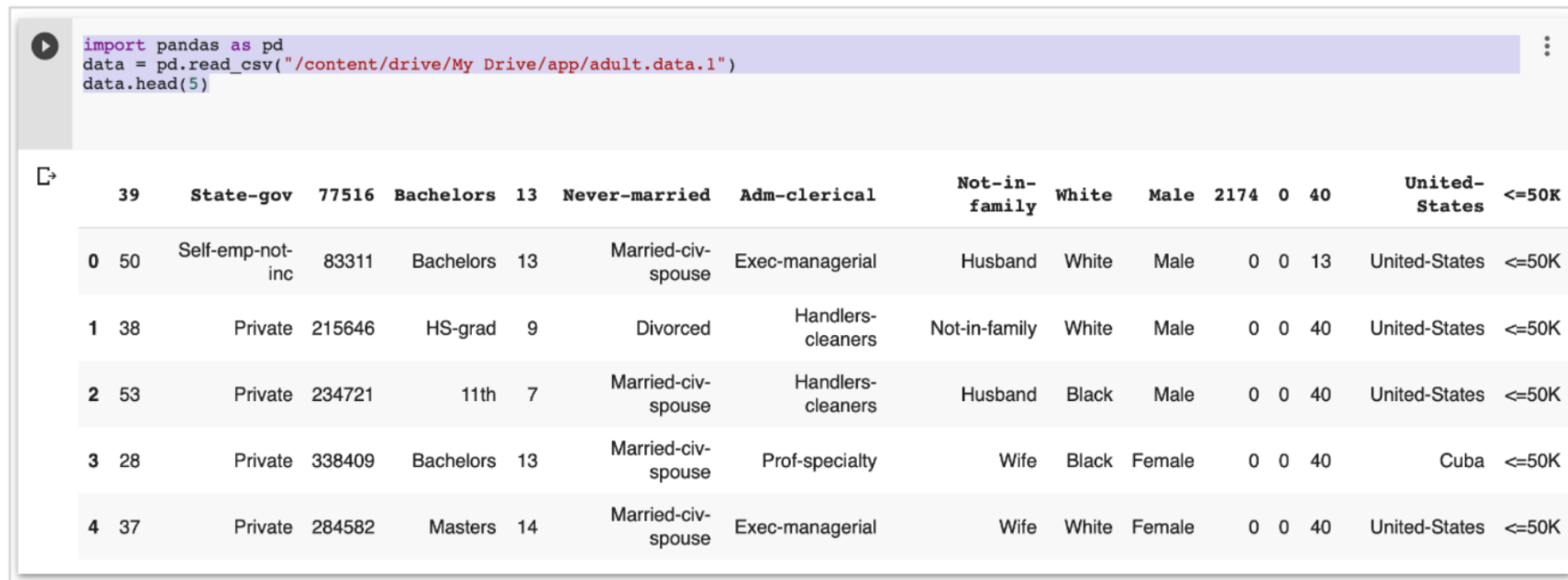
As the message says, the **adult.data.1** file is now added to your drive. You can verify this by examining the folder contents of your drive. Alternatively, type in the following code in a new Code cell:

```
import pandas as pd

data = pd.read_csv("/content/drive/My Drive/app/adult.data.1")

data.head(5)
```

Run the code now and you will see the following output:



The screenshot shows a Jupyter Notebook interface. At the top, a code cell contains the following Python code:

```
import pandas as pd
data = pd.read_csv("/content/drive/My Drive/app/adult.data.1")
data.head(5)
```

Below the code cell, the output is displayed as a table. The table has 17 columns and 5 rows of data. The first row is a header row, and the subsequent four rows are data rows indexed 0 through 4.

	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
0	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
1	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
2	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
3	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K
4	37	Private	284582	Masters	14	Married-civ-spouse	Exec-managerial	Wife	White	Female	0	0	40	United-States	<=50K

Cloning Git Repository

You can clone the entire GitHub repository into Colab using the **git** command. For example, to clone the keras tutorial, type the following command in the Code cell:

```
!git clone https://github.com/wxs/keras-mnist-tutorial.git
```

After a successful run of the command, you would see the following output:

```
Cloning into 'keras-mnist-tutorial'...
remote: Enumerating objects: 26, done.
remote: Total 26 (delta 0), reused 0 (delta 0), pack-reused 26
Unpacking objects: 100% (26/26), done.
```

Once the repo is cloned, locate a Jupyter project (e.g. MNIST in keras.ipynb) in it, right-click on the file name and select **Open With / Colaboratory** menu option to open the project in Colab.

System Aliases

To get a list of shortcuts for common operations, execute the following command:

```
!ls /bin
```

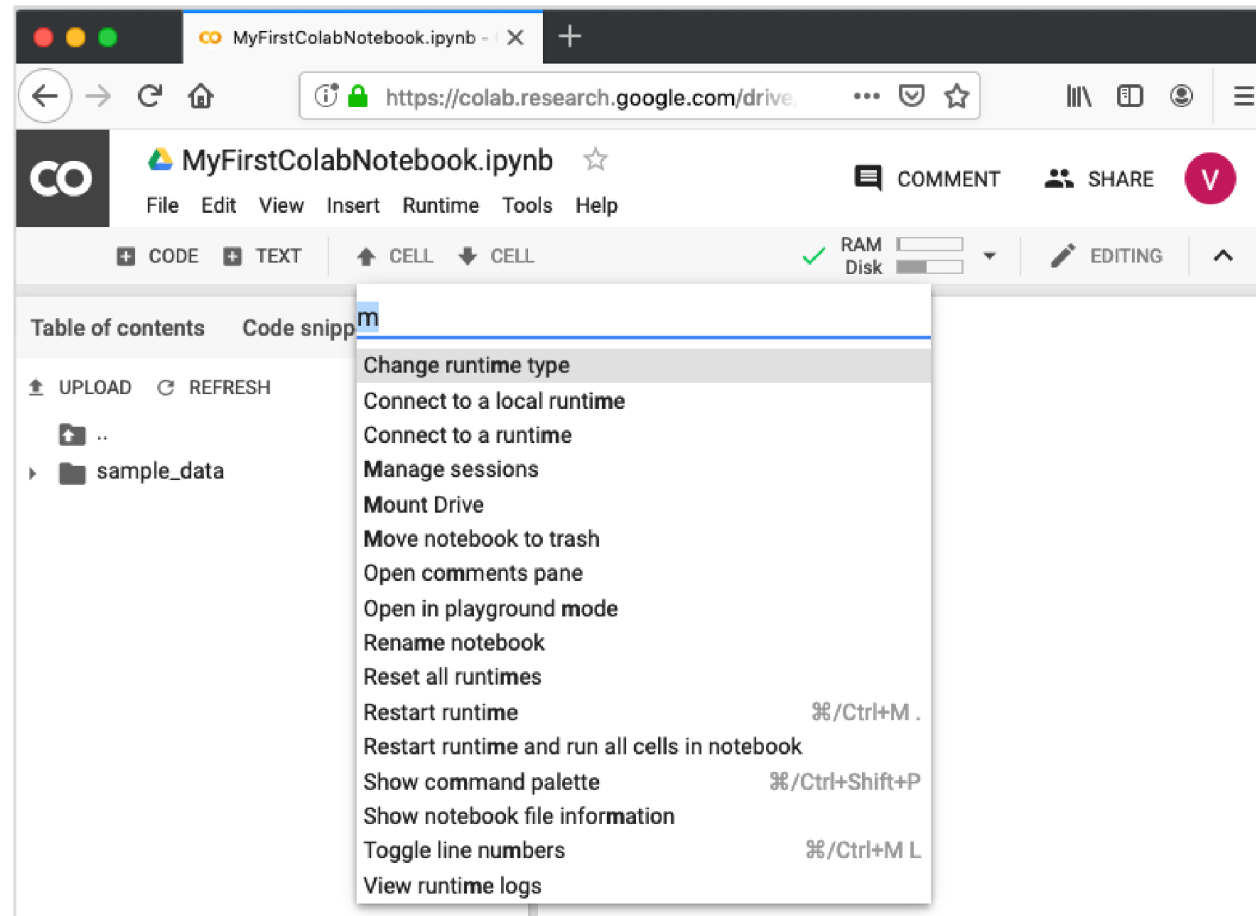

Executing External Python Files

Mounting Drive

First, you need to mount your Google Drive in Colab. Select the following menu options:

Tools / Command palette

You will see the list of commands as shown in this screenshot:



Type a few letters like "m" in the search box to locate the mount command. Select **Mount Drive** command from the list. The following code would be inserted in your Code cell.

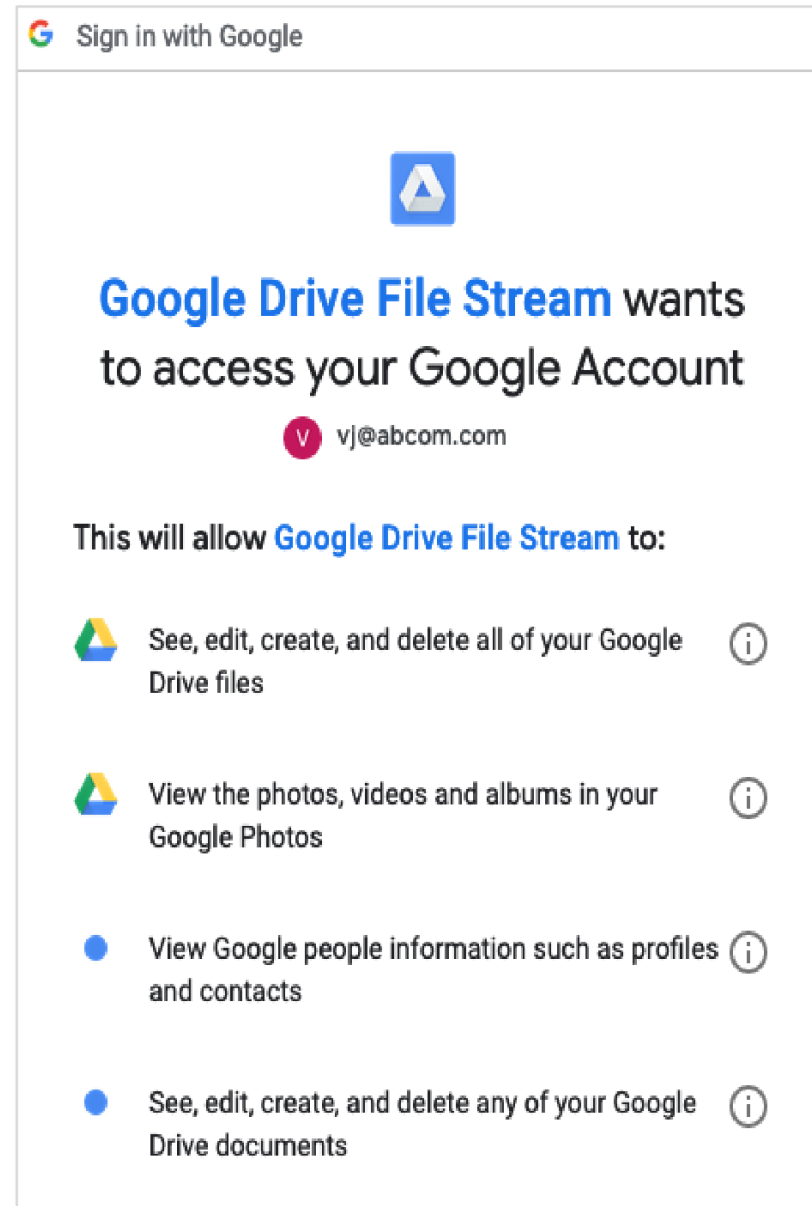
```
# Run this cell to mount your Google Drive.  
  
from google.colab import drive  
  
drive.mount('/content/drive')
```

If you run this code, you will be asked to enter the authentication code. The corresponding screen looks as shown below:

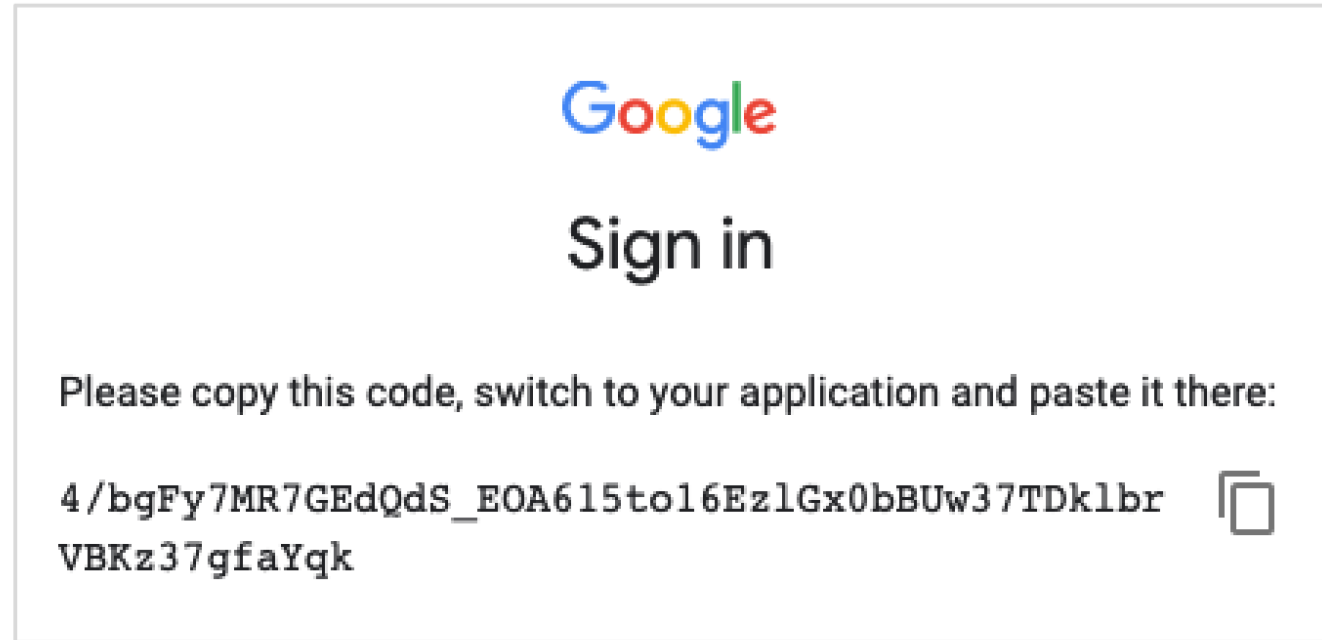


The screenshot shows a Google Colab interface. At the top, a code cell contains the following text: `# Run this cell to mount your Google Drive.` followed by `from google.colab import drive` and `drive.mount('/content/drive')`. Below the code cell, a message reads: "... Go to this URL in a browser: https://accounts.google.com/o/oauth2/auth?client_id=947318989803-6bn6qk8gdqf4n4g3pfee6491hc0". Below the URL, it says "Enter your authorization code:" followed by a text input field.

Open the above URL in your browser. You will be asked to login to your Google account. Now, you will see the following screen:



If you grant the permissions, you will receive your code as follows:



Cut-n-paste this code in the Code cell and hit ENTER. After a while, the drive will be mounted as seen in the screenshot below:



Now, you are ready to use the contents of your drive in Colab.

Listing Drive Contents

You can list the contents of the drive using the **ls** command as follows:

```
!ls "/content/drive/My Drive/Colab Notebooks"
```

This command will list the contents of your Colab Notebooks folder. The sample output of my drive contents are shown here:

```
Greeting.ipynb  
hello.py  
LogisticRegressionCensusData.ipynb  
LogisticRegressionDigitalOcean.ipynb  
MyFirstColabNotebook.ipynb  
SamplePlot.ipynb
```

Running Python Code

Now, let us say that you want to run a Python file called **hello.py** stored in your Google Drive. Type the following command in the Code cell:

```
!python3 "/content/drive/My Drive/Colab Notebooks/hello.py"
```

The contents of hello.py are given here for your reference:

```
print("Welcome to TutorialsPoint!")
```

You will now see the following output:

```
Welcome to TutorialsPoint!
```

Besides the text output, Colab also supports the graphical outputs. We will see this in the next chapter.

Graphical Outputs

Colab also supports rich outputs such as charts. Type in the following code in the Code cell.

```
import numpy as np
from matplotlib import pyplot as plt

y = np.random.randn(100)
x = [x for x in range(len(y))]

plt.plot(x, y, '-')
plt.fill_between(x, y, 200, where=(y > 195), facecolor='g', alpha=0.6)

plt.title("Sample Plot")
plt.show()
```

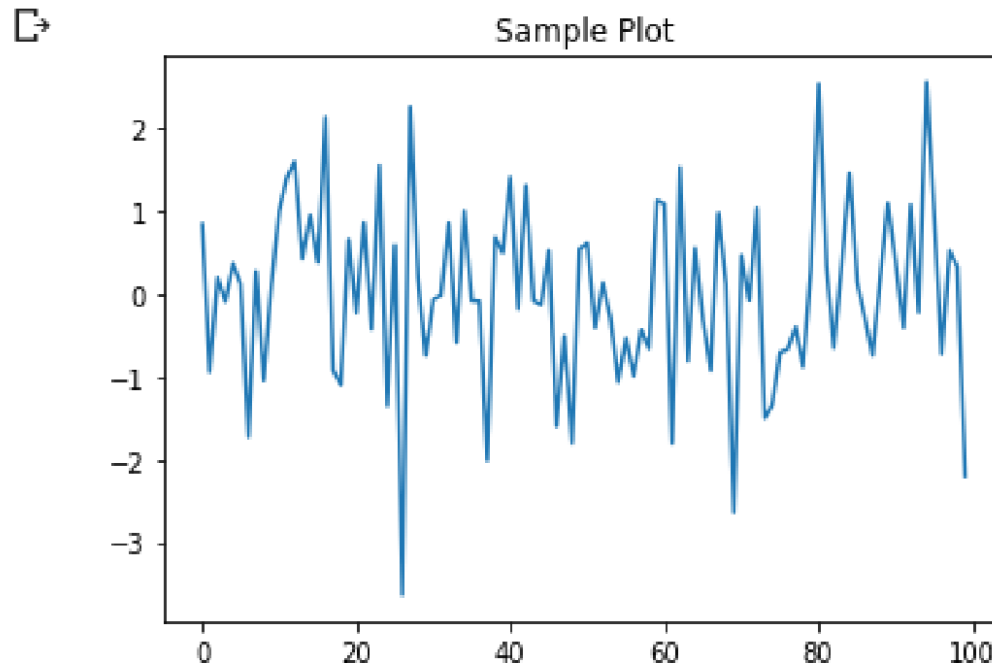
Now, if you run the code, you will see the following output:

```
import numpy as np
from matplotlib import pyplot as plt

y = np.random.randn(100)
x = [x for x in range(len(y))]

plt.plot(x, y, '-')
plt.fill_between(x, y, 200, where=(y > 195), facecolor='g', alpha=0.6)

plt.title("Sample Plot")
plt.show()
```



Code Editing Help

Function List

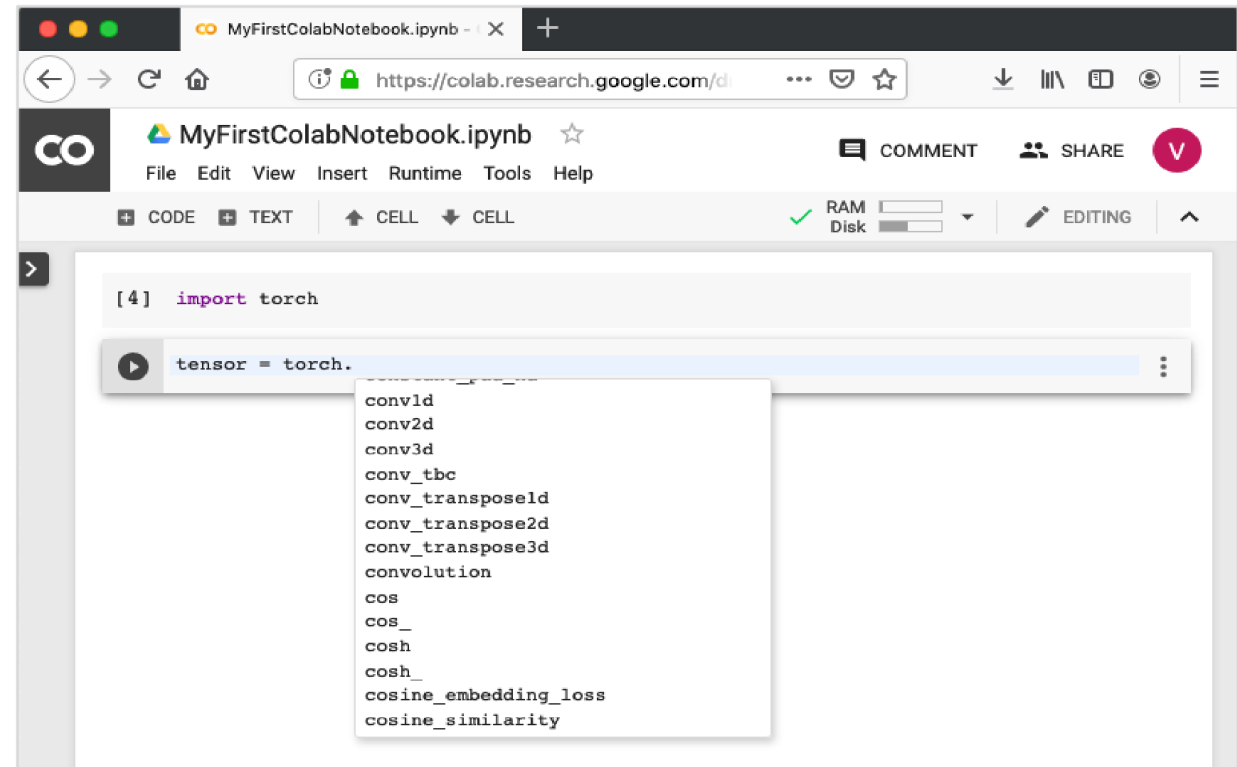
Step 1: Open a new notebook and type in the following code in the Code cell:

```
import torch
```

Step 2: Run the code by clicking on the **Run** icon in the left panel of the Code cell. Add another Code cell and type in the following code:

```
Tensor = torch.
```

At this point, suppose you have forgotten what are the various functions available in **torch** module. You can ask for the context-sensitive help on function names by hitting the **TAB** key. Note the presence of the **DOT** after the **torch** keyword. Without this DOT, you will not see the context help. Your screen would look like as shown in the screenshot here:



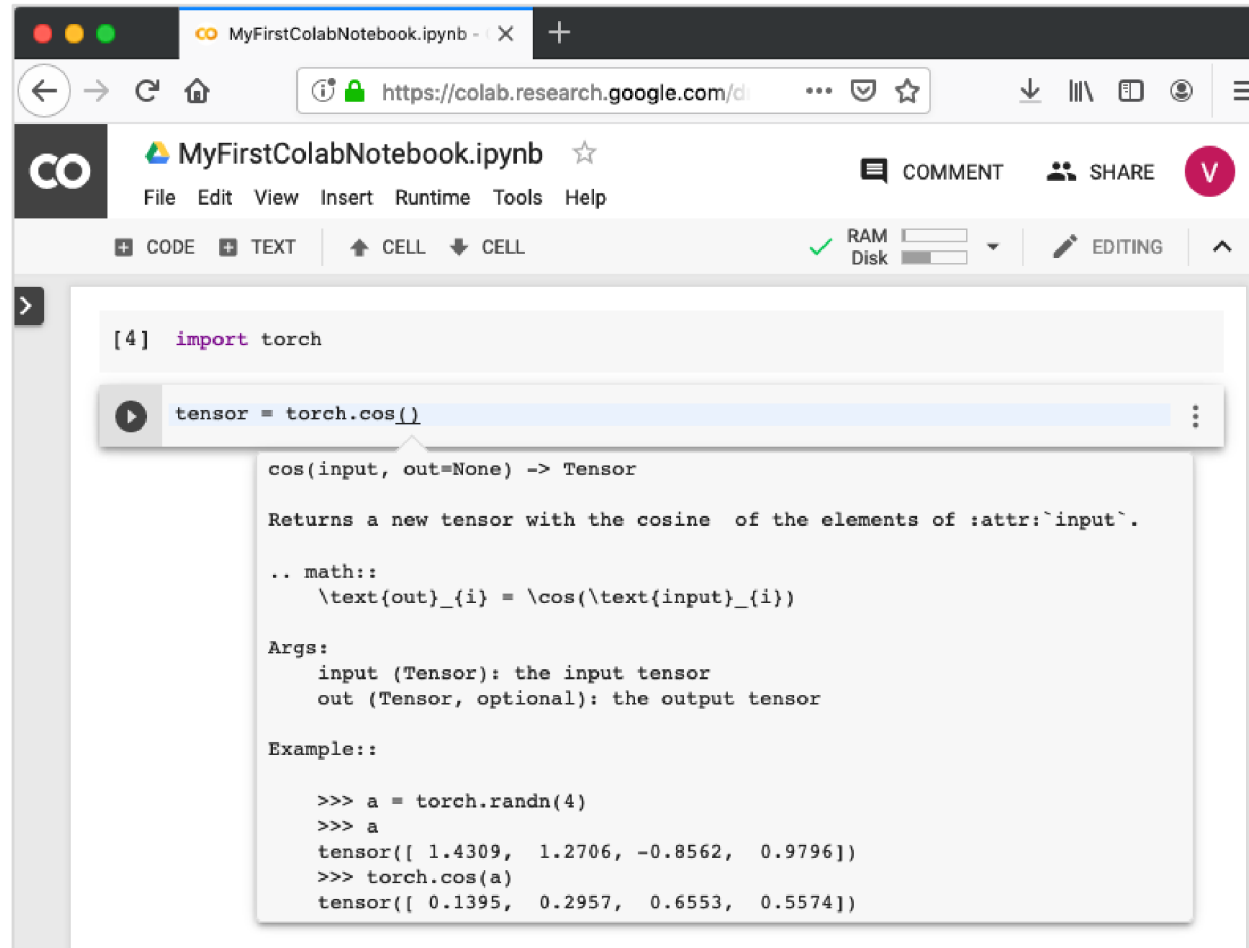
Function Documentation

Colab gives you the documentation on any **function** or **class** as a context-sensitive help.

Type the following code in your code window:

```
Tensor = torch.cos(
```

Now, hit **TAB** and you will see the documentation on **cos** in the popup window as shown in the screenshot here. Note that you need to type in **open parenthesis** before hitting TAB.



Magics

Magics is a set of system commands that provide a mini extensive command language.

Magics are of two types:

- Line magics
- Cell magics

The line magics as the name indicates that it consists of a single line of command, while the cell magic covers the entire body of the code cell.

In case of line magics, the command is prepended with a single % character and in the case of cell magics, it is prepended with two % characters (%%).

Let us look into some examples of both to illustrate these.

Line Magics

Type the following code in your code cell:

```
%ldir
```

You will see the contents of your local directory, something like this -

```
drwxr-xr-x 3 root  4096 Jun 20 10:05 drive/  
drwxr-xr-x 1 root  4096 May 31 16:17 sample_data/
```

Try the following command:

```
%history
```

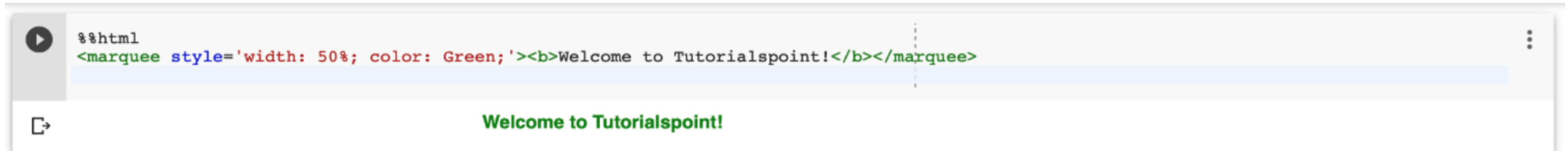
This presents the complete history of commands that you have previously executed.

Cell Magics

Type in the following code in your code cell:

```
%%html  
  
<marquee style='width: 50%; color: Green; '><b>Welcome to  
Tutorialspoint!</b></marquee>
```

Now, if you run the code and you will see the scrolling welcome message on the screen as shown here:



The screenshot shows a Jupyter Notebook interface. At the top, there is a code editor with the following text: `%%html` followed by a blank line, and then `<marquee style='width: 50%; color: Green; '><b>Welcome to Tutorialspoint!</b></marquee>`. The code is syntax-highlighted. Below the code editor, there is a toolbar with a play button (run) and a copy icon. To the right of the toolbar, the output of the code is displayed: **Welcome to Tutorialspoint!** in green text. The output is centered and has a green background.

The following code will add SVG to your document.

```
%%html
```

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 600 400" width="400"  
height="400">
```

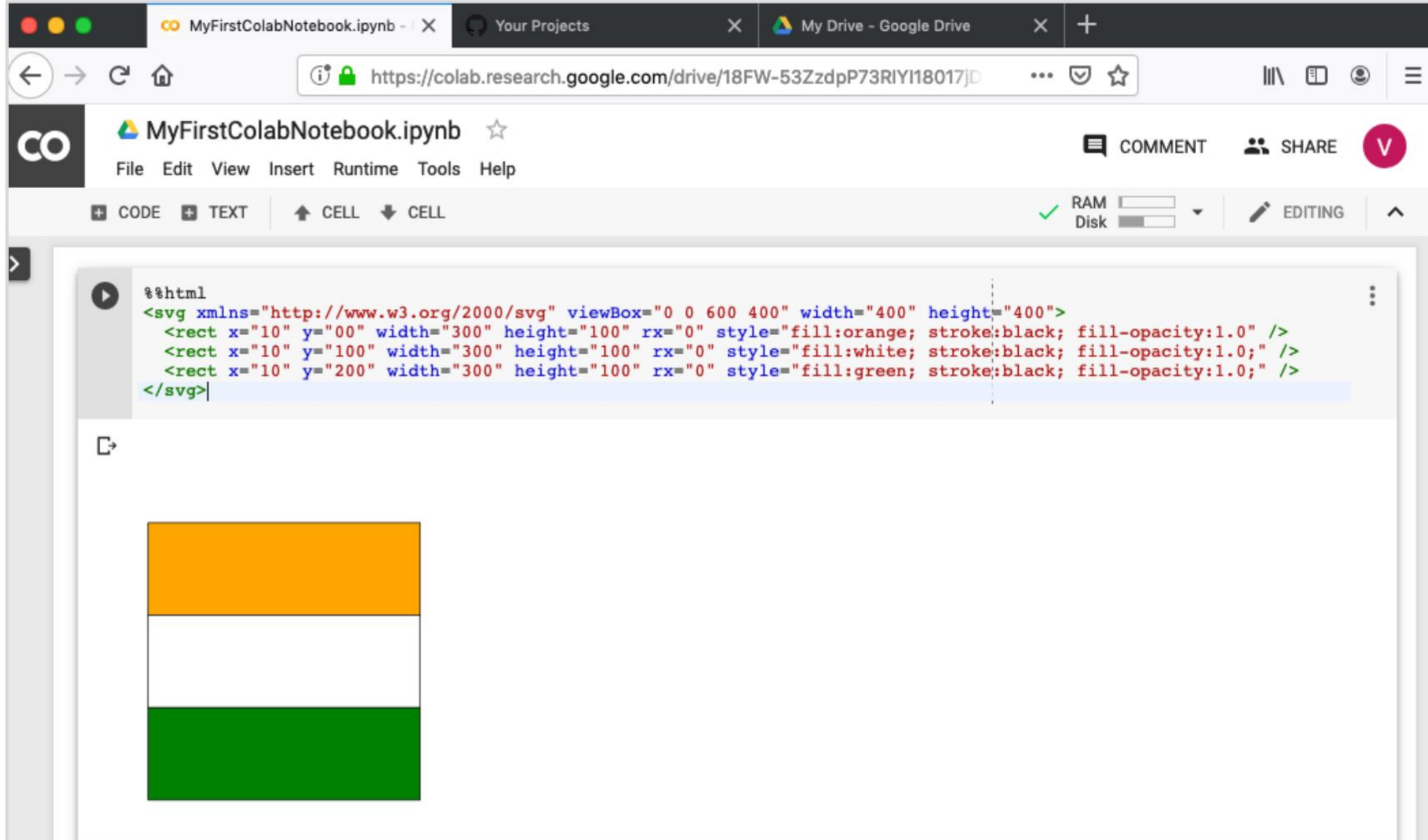
```
  <rect x="10" y="00" width="300" height="100" rx="0" style="fill:orange;  
stroke:black; fill-opacity:1.0" />
```

```
  <rect x="10" y="100" width="300" height="100" rx="0" style="fill:white;  
stroke:black; fill-opacity:1.0;" />
```

```
  <rect x="10" y="200" width="300" height="100" rx="0" style="fill:green;  
stroke:black; fill-opacity:1.0;" />
```

```
</svg>
```

If you run the code, you would see the following output:



The screenshot shows a Google Colab notebook interface. The browser tabs at the top include 'MyFirstColabNotebook.ipynb', 'Your Projects', and 'My Drive - Google Drive'. The address bar shows the URL 'https://colab.research.google.com/drive/18FW-53ZzdpP73RIYI18017JD'. The notebook title is 'MyFirstColabNotebook.ipynb'. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. The toolbar shows 'CODE', 'TEXT', 'CELL', and 'CELL' buttons, along with RAM and Disk usage indicators and an 'EDITING' button. The code cell contains the following SVG code:

```
%%html
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 600 400" width="400" height="400">
  <rect x="10" y="00" width="300" height="100" rx="0" style="fill:orange; stroke:black; fill-opacity:1.0" />
  <rect x="10" y="100" width="300" height="100" rx="0" style="fill:white; stroke:black; fill-opacity:1.0;" />
  <rect x="10" y="200" width="300" height="100" rx="0" style="fill:green; stroke:black; fill-opacity:1.0;" />
</svg>
```

The output of the code is a vertical rectangle divided into three horizontal bands of equal height: orange at the top, white in the middle, and green at the bottom, representing the Indian national flag.

Magics List

To get a complete list of supported magics, execute the following command:

```
%lsmagic
```

You will see the following output:

```
Available line magics:
```

```
%alias    %alias_magic %autocall %automagic %autosave %bookmark %cat %cd  
%clear    %colors    %config %connect_info %cp %debug %dhist %dirs  
%doctest_mode %ed %edit %env %gui %hist %history %killbgscripts %ldir  
           %less %lf %lk %ll %load %load_ext %loadpy %logoff %logon
```


Adding Forms

Colab provides a very useful utility called Forms that allows you to accept inputs from the user at runtime. Let us now move on to see how to add forms to your notebook.

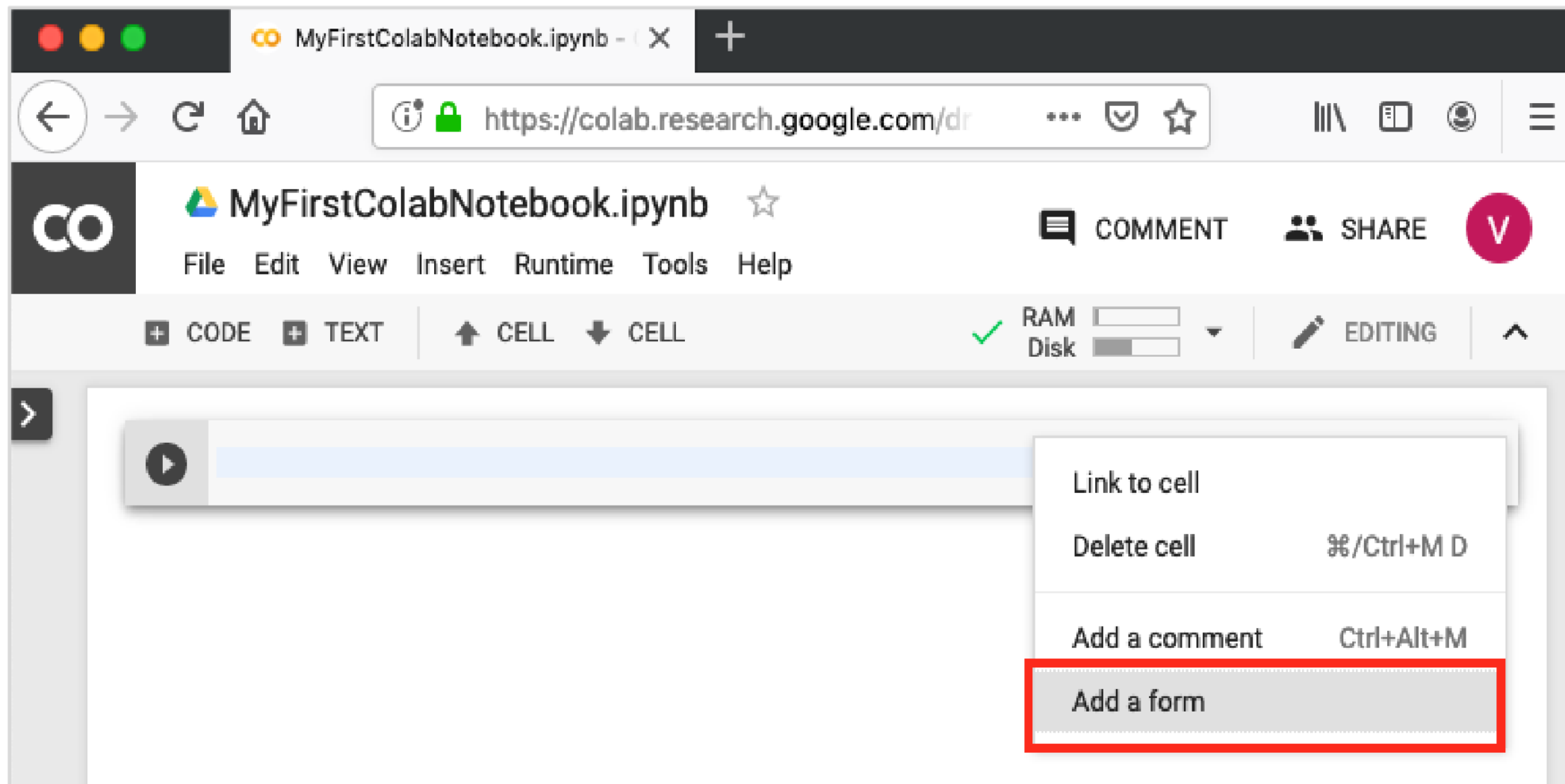
Adding Form

In an earlier lesson, you used the following code to create a time delay:

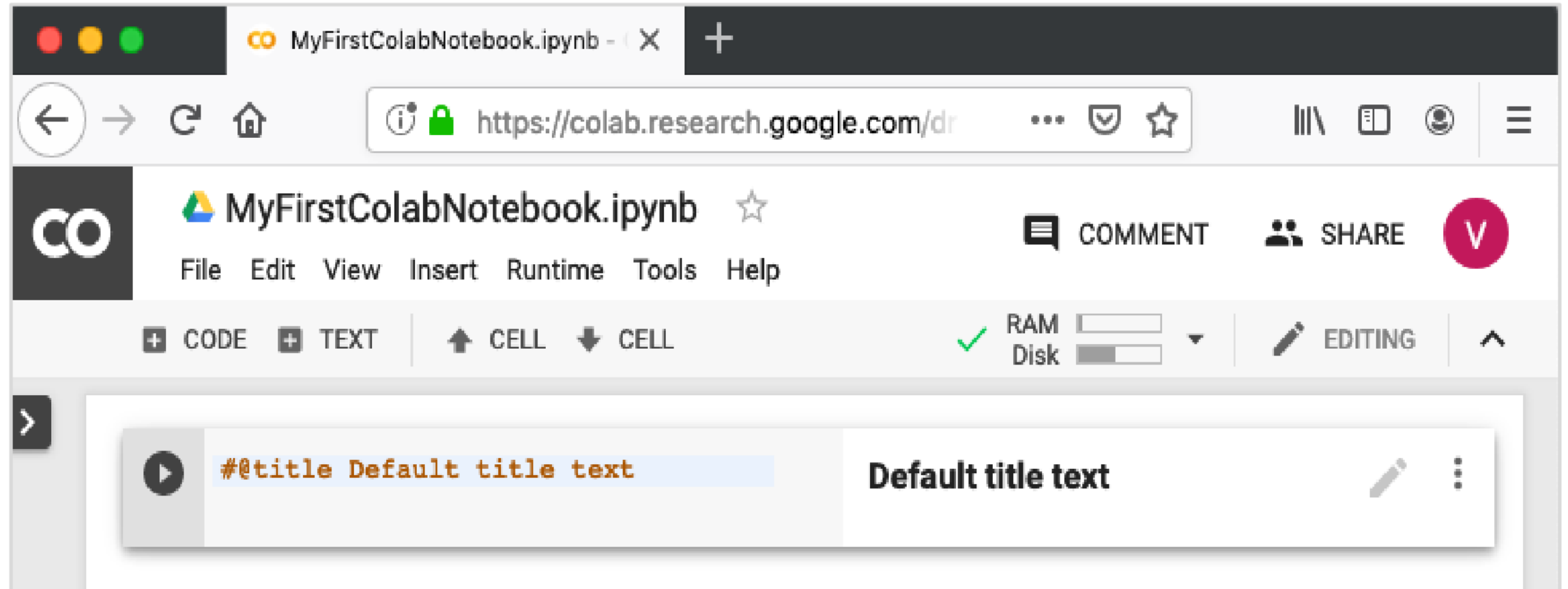
```
import time  
print(time.ctime())  
time.sleep(5)  
print (time.ctime())
```

Suppose, you want a user set time delay instead of a fixed delay of 5 seconds. For this, you can add a Form to the Code cell to accept the sleep time.

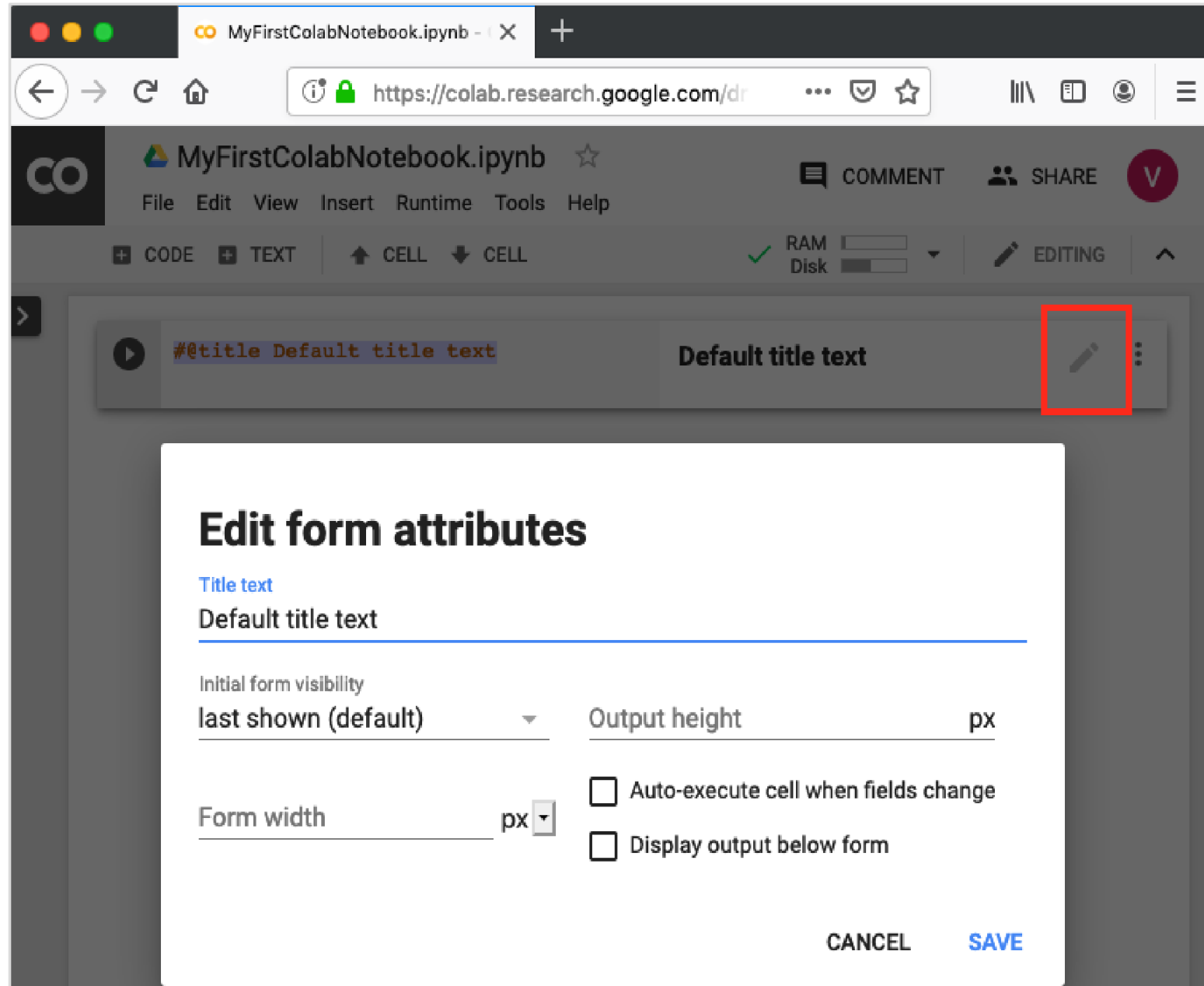
Open a new notebook. Click on the **Options** (vertically-dotted) menu. A popup menu shows up as seen in the screenshot below:



Now, select **Add a form** option. It will add the form to your Code cell with a Default title as seen in the screenshot here:

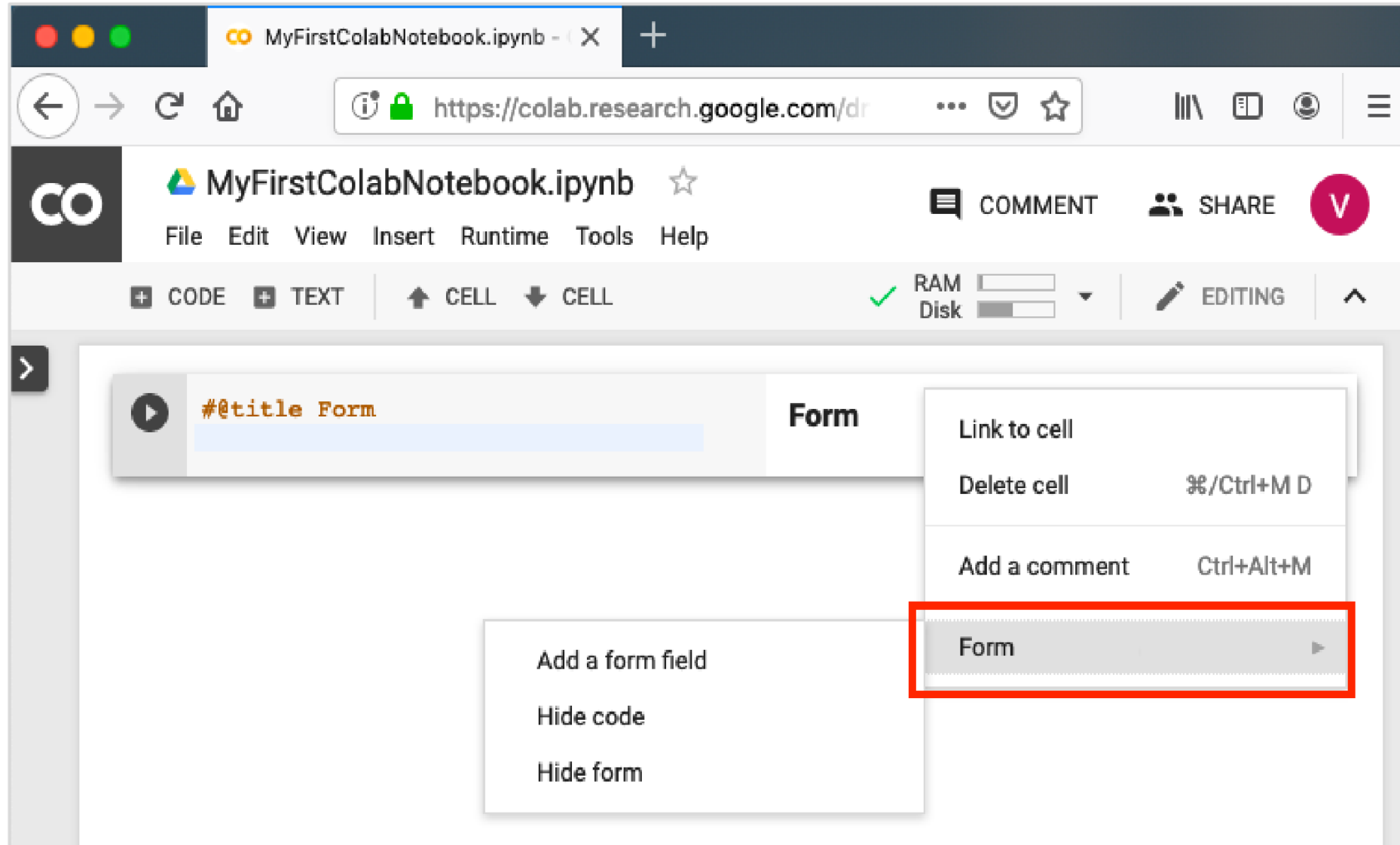


To change the title of the form, click on the **Settings** button (pencil icon on the right). It will pop up a settings screen as shown here:



Adding Form Fields

To add a form field, click the **Options** menu in the Code cell, click on the **Form** to reveal the submenus. The screen will look as shown below:



Select **Add a form field** menu option. A dialog pops up as seen here:

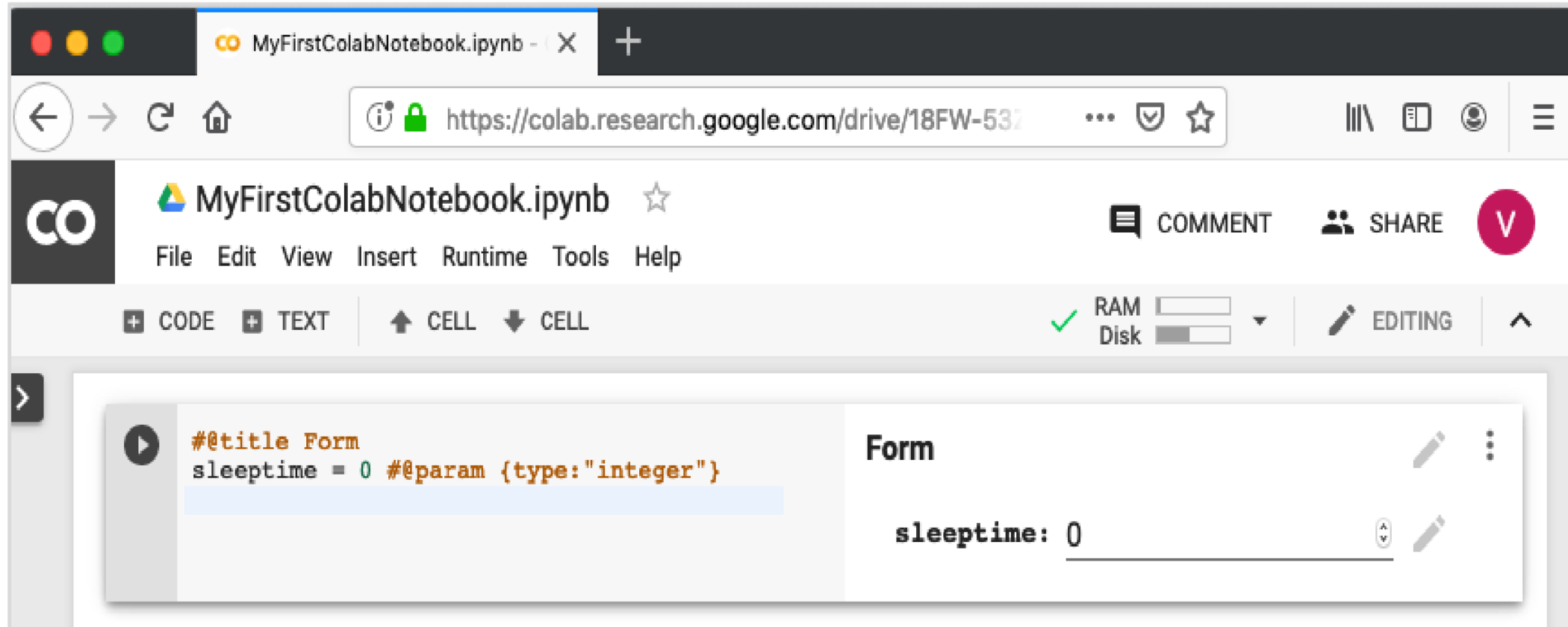
The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL `https://colab.research.google.com/drive/18FW-532`. The notebook title is `MyFirstColabNotebook.ipynb`. The menu bar includes `File`, `Edit`, `View`, `Insert`, `Runtime`, `Tools`, and `Help`. The toolbar shows `CODE`, `TEXT`, `CELL`, and `CELL` options. The RAM and Disk usage is shown as `RAM Disk` with a green checkmark. The `EDITING` mode is active. The notebook content shows a `#@title Form` cell. A dialog box titled `Add new form field` is open, displaying the following fields:

- `Form field type` dropdown menu with `input` selected.
- `Variable type` dropdown menu with `string` selected.
- `Variable name` text input field with `variable_name` entered.

The dialog has `CANCEL` and `SAVE` buttons at the bottom right.

Leave the **Form field type** to **input**. Change the **Variable name** to **sleeptime** and set the **Variable type** to **integer**. Save the changes by clicking the **Save** button.

Your screen will now look like the following with the **sleeptime** variable added into the code.



The screenshot shows the Google Colab web interface. The browser tab is titled "MyFirstColabNotebook.ipynb". The address bar shows the URL "https://colab.research.google.com/drive/18FW-532". The Colab logo is in the top left. The top navigation bar includes "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". On the right, there are "COMMENT" and "SHARE" buttons, and a user profile icon. Below the navigation bar, there are tabs for "CODE" and "TEXT", and buttons for "CELL" (up and down arrows). A status bar shows "RAM" and "Disk" usage, a green checkmark, and "EDITING" mode. The main content area shows a code cell with the following code:

```
##title Form
sleeptime = 0 ##param {type:"integer"}
```

To the right of the code cell, a form preview is shown. It has a title "Form" and a single input field labeled "sleeptime: 0". The input field is a text box with a value of "0".

Next, let us see how to test the form by adding some code that uses the **sleeptime** variable.

Testing Form

Add a new Code cell underneath the form cell. Use the code given below:

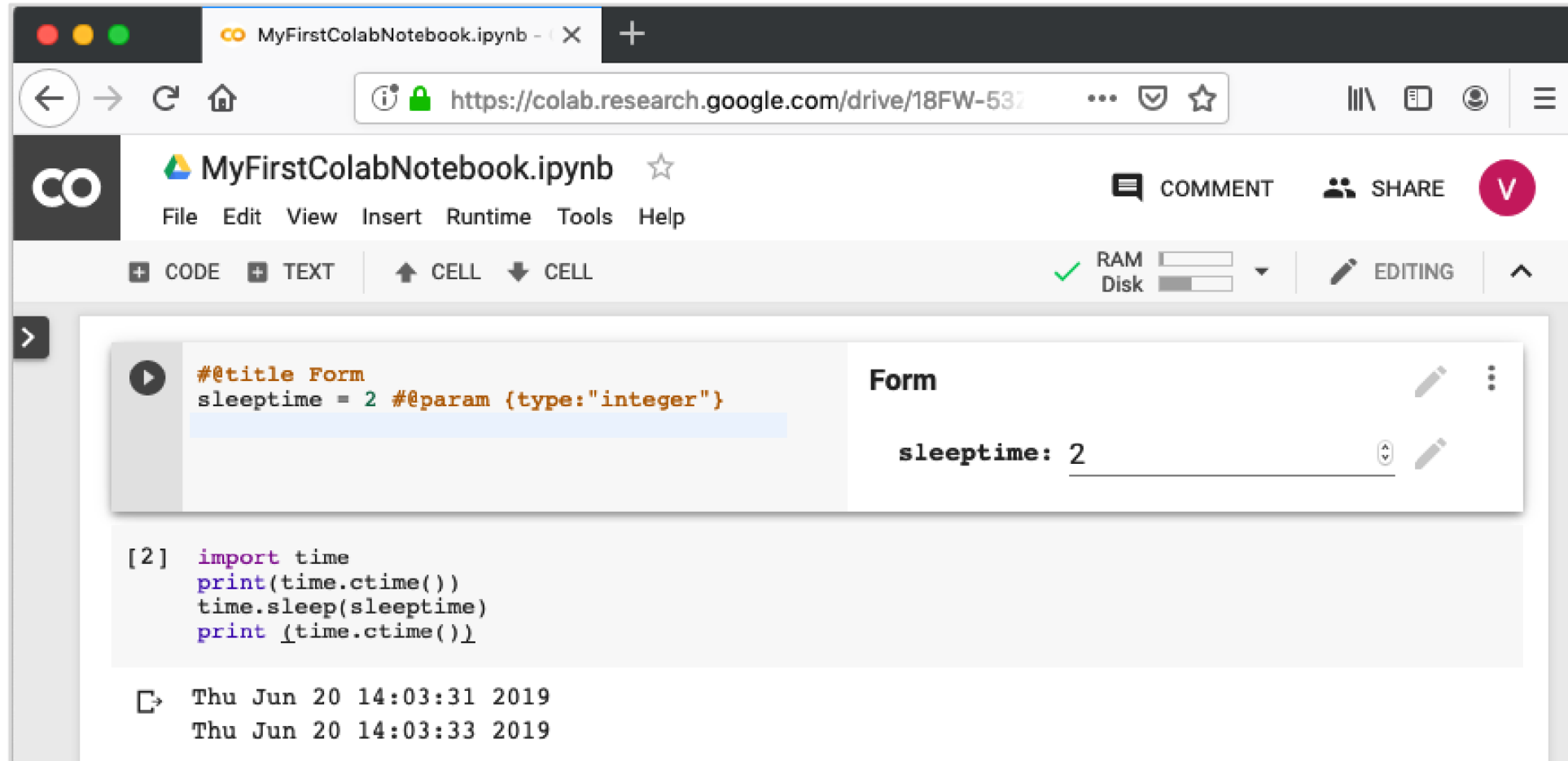
```
import time  
  
print(time.ctime())  
  
time.sleep(sleeptime)  
  
print (time.ctime())
```

You have used this code in the earlier lesson. It prints the current time, waits for a certain amount of time and prints a new timestamp. The amount of time that the program waits is set in the variable called **sleeptime**.

Now, go back to the **Form** Cell and type in a value of 2 for the **sleeptime**. Select the following menu:

```
Runtime / Run all
```


This runs the entire notebook. You can see an output screen as shown below.



The screenshot displays a Google Colab notebook titled "MyFirstColabNotebook.ipynb". The browser address bar shows the URL <https://colab.research.google.com/drive/18FW-532>. The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with options for CODE, TEXT, and CELL. A RAM and Disk usage indicator is visible, showing a green checkmark and a progress bar. The notebook content consists of two cells. The first cell is a form titled "Form" with a parameter "sleeptime" set to 2. The second cell contains Python code that imports the time module, prints the current time, sleeps for the specified duration, and prints the time again. The output of the second cell shows the execution time as "Thu Jun 20 14:03:31 2019" and "Thu Jun 20 14:03:33 2019".

```
#@title Form
sleeptime = 2 #@param {type:"integer"}

[2] import time
    print(time.ctime())
    time.sleep(sleeptime)
    print (time.ctime())

Thu Jun 20 14:03:31 2019
Thu Jun 20 14:03:33 2019
```

Notice that it has taken your input value of 2 for the **sleeptime**. Try changing this to a different value and **Run all** to see its effect.

Inputting Text

To accept a text input in your form, enter the following code in a new code cell.

```
name = 'Tutorialspoint' #@param {type:"string"}  
print(name)
```

Now, if you run the Code cell, whatever the name you set in the form would be printed on the screen. By default, the following output would appear on the screen.

```
Tutorialspoint
```

Note that you may use the menu options as shown for the integer input to create a **Text** input field.

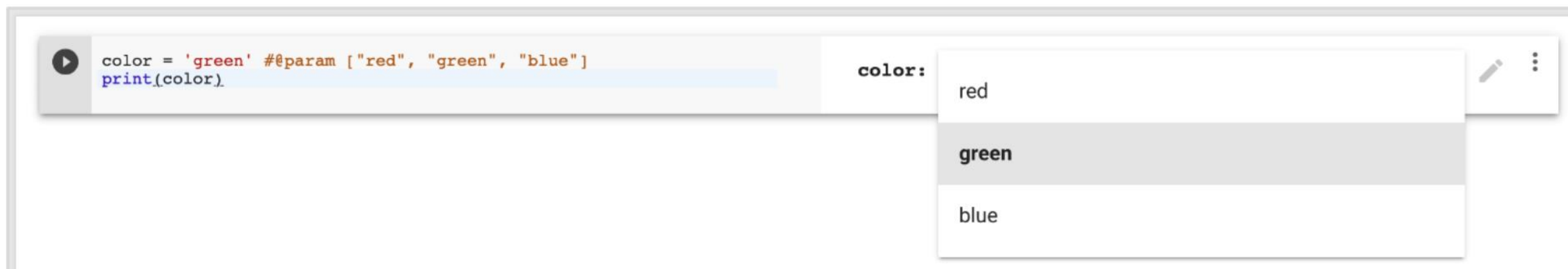
Dropdown List

To add a dropdown list to your form, use the following code:

```
color = 'green' #@param ["red", "green", "blue"]  
print(color)
```

This creates a dropdown list with three values - red, green and blue. The default selection is green.

The dropdown list is shown in the screenshot below:



Date Input

Colab Form allows you to accept dates in your code with validations. Use the following code to input date in your code.

```
#@title Date fields  
date_input = '2019-06-03' #@param {type:"date"}  
print(date_input)
```

The Form screen looks like the following.



The screenshot shows the Colab Form interface. On the left, a code cell is displayed with the following code: `#@title Date fields` and `variable_name = "2019-06-20" #@param {type:"date"}`. The code is highlighted in blue. On the right, the form is titled "Date fields". Below the title, the variable `variable_name` is shown with a date input field. The date is set to "2019 / 6 / 20". The input field has a calendar icon and a pencil icon for editing. There are also up and down arrows for each part of the date (year, month, day).

Try inputting a wrong date value and observe the validations.

So far you have learned how to use Colab for creating and executing Jupyter notebooks with your Python code. In the next chapter, we will see how to install popular ML libraries in your notebook so that you can use those in your Python code.

Installing ML Libraries

Colab supports most of machine learning libraries available in the market. In this chapter, let us take a quick overview of how to install these libraries in your Colab notebook.

To install a library, you can use either of these options:

```
!pip install
```

or

```
!apt-get install
```

Keras

Keras, written in Python, runs on top of TensorFlow, CNTK, or Theano. It enables easy and fast prototyping of neural network applications. It supports both convolutional networks (CNN) and recurrent networks, and also their combinations. It seamlessly supports GPU.

To install Keras, use the following command:

```
!pip install -q keras
```

PyTorch

PyTorch is ideal for developing deep learning applications. It is an optimized tensor library and is GPU enabled. To install PyTorch, use the following command:

```
!pip3 install torch torchvision
```

MxNet

Apache MxNet is another flexible and efficient library for deep learning. To install MxNet execute the following commands:

```
!apt install libnVRTC8.0  
!pip install mxnet-cu80
```

OpenCV

OpenCV is an open source computer vision library for developing machine learning applications. It has more than 2500 optimized algorithms which support several applications such as recognizing faces, identifying objects, tracking moving objects, stitching images, and so on. Giants like Google, Yahoo, Microsoft, Intel, IBM, Sony, Honda, Toyota use this library. This is highly suited for developing real-time vision applications.

To install OpenCV use the following command:

```
!apt-get -qq install -y libsm6 libxext6 && pip install -q -U opencv-python
```


XGBoost

XGBoost is a distributed gradient boosting library that runs on major distributed environments such as Hadoop. It is highly efficient, flexible and portable. It implements ML algorithms under the Gradient Boosting framework.

To install XGBoost, use the following command:

```
!pip install -q xgboost==0.4a30
```

GraphViz

Graphviz is an open source software for graph visualizations. It is used for visualization in networking, bioinformatics, database design, and for that matter in many domains where a visual interface of the data is desired.

To install GraphViz, use the following command:

```
!apt-get -qq install -y graphviz && pip install -q pydot
```

By this time, you have learned to create Jupyter notebooks containing popular machine learning libraries. You are now ready to develop your machine learning models. This requires high processing power. Colab provides free GPU for your notebooks.

Using Free GPU

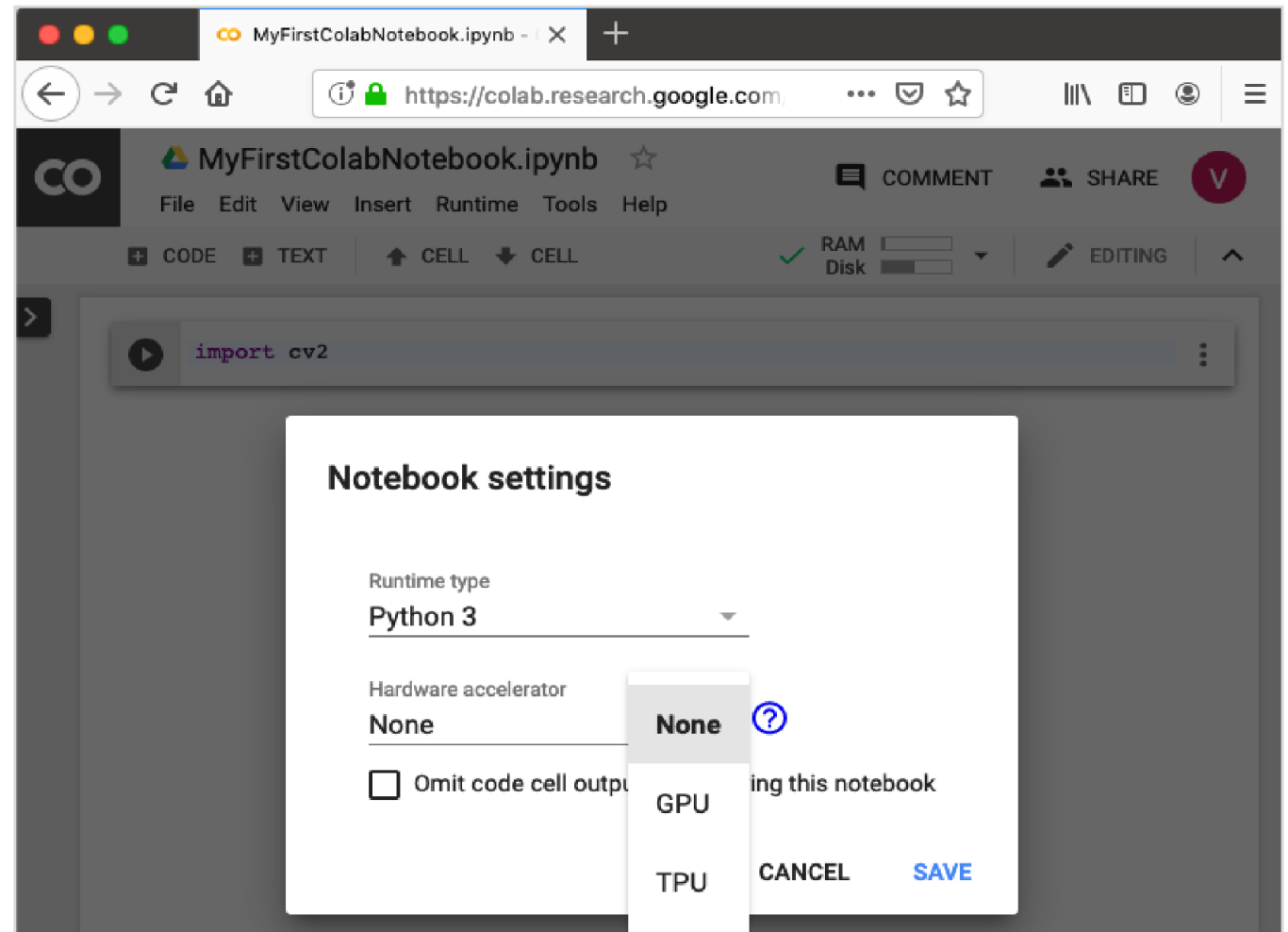
Google provides the use of free GPU for your Colab notebooks.

Enabling GPU

To enable GPU in your notebook, select the following menu options:

Runtime / Change runtime type

You will see the following screen as the output:



Select **GPU** and your notebook would use the free GPU provided in the cloud during processing. To get the feel of GPU processing, try running the sample application from **MNIST** tutorial that you cloned earlier.

```
!python3 "/content/drive/My Drive/app/mnist_cnn.py"
```

Try running the same Python file without the GPU enabled. Did you notice the difference in speed of execution?

Testing for GPU

You can easily check if the GPU is enabled by executing the following code:

```
import tensorflow as tf  
tf.test.gpu_device_name()
```

If the GPU is enabled, it will give the following output:

```
'/device:GPU:0'
```

Listing Devices

If you are curious to know the devices used during the execution of your notebook in the cloud, try the following code:

```
from tensorflow.python.client import device_lib  
  
device_lib.list_local_devices()
```

You will see the output as follows:

```
[name: "/device:CPU:0"  
  device_type: "CPU"  
  memory_limit: 268435456  
  locality {  
  }
```

Checking RAM

To see the memory resources available for your process, type the following command:

```
!cat /proc/meminfo
```

You will see the following output:

```
MemTotal:      13335276 kB
MemFree:       7322964 kB
MemAvailable:  10519168 kB
Buffers:       95732 kB
Cached:        2787632 kB
SwapCached:    0 kB
Active:        2433984 kB
Inactive:      3060124 kB
Active(anon):  2101704 kB
Inactive(anon): 22880 kB
Active(file):  332280 kB
Inactive(file): 3037244 kB
```

Running Neural Style Transfer

Now going back to our original neural style transfer project, <https://github.com/anishathalye/neural-style>, let's say you want to try running this on Colab. There are a few steps to follow:

Step 1: Downloading the Project

By default, each cell in Colab runs Python code. However, we can also run special commands to download projects and install packages using a `!` before the command. For example, to download the project, you would run:

```
[1] !git clone https://github.com/anishathalye/neural-style.git
```

```
Cloning into 'neural-style'...
remote: Enumerating objects: 292, done.
remote: Total 292 (delta 0), reused 0 (delta 0), pack-reused 292
Receiving objects: 100% (292/292), 2.63 MiB | 2.15 MiB/s, done.
Resolving deltas: 100% (158/158), done.
```

Now that the project has been downloaded, you want to enter that project directory. You can do so using the **cd** command, which in colab requires a % before it:

```
[4] %cd neural-style
```

```
↳ /content/neural-style
```

The outputted */content/neural-style* is the new directory you are in.

Step 2: Installing the Package Requirements

Generally Python projects come with *requirements.txt* files that indicate which packages that codebase depends on. You can see this file using the *cat* command, as shown below:

```
[5] !cat requirements.txt
```

```
↳ numpy  
Pillow # provides PIL  
scipy==1.1  
tensorflow-gpu >= 1.0 # installs tensorflow with GPU support, should still work even without GPU
```

This project depends on numpy (for math operations), Pillow (for manipulating images), scipy (for scientific operations), and tensorflow-gpu (for training neural networks on the GPU). You can install these packages by running the following. You might get some warnings/errors but you can ignore those.

```
▶ !pip install -r requirements.txt
```

Now as one final step, you'll need to install tensorflow 1.13.2. In the requirements file, the author lists numpy, Pillow, scipy==1.1, and any version of tensorflow >= 1.0. Just recently a new version of tensorflow was released (2.0) and this codebase no longer works with 2.0. To install the right version of tensorflow, run:

```
[8] !pip install tensorflow-gpu==1.13.2
```

With that step complete, all the packages and source code you need to run the project should be downloaded and you should be ready to go.

Step 2: Downloading the Pretrained Model

This repo contains a pretrained model called VGG that you will need to download before running. You can do so using the *wget* command, as shown below:

```
[4] !wget http://www.vlfeat.org/matconvnet/models/imagenet-vgg-verydeep-19.mat
```

```
[->] --2019-10-04 23:22:53-- http://www.vlfeat.org/matconvnet/models/imagenet-vgg-verydeep-19.mat
Resolving www.vlfeat.org (www.vlfeat.org)... 64.90.48.57
Connecting to www.vlfeat.org (www.vlfeat.org)|64.90.48.57|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 534904783 (510M)
Saving to: 'imagenet-vgg-verydeep-19.mat'

imagenet-vgg-veryde 100%[=====>] 510.12M  21.2MB/s   in 24s

2019-10-04 23:23:17 (21.2 MB/s) - 'imagenet-vgg-verydeep-19.mat' saved [534904783/534904783]
```

Step 3: Running and Visualizing the Output

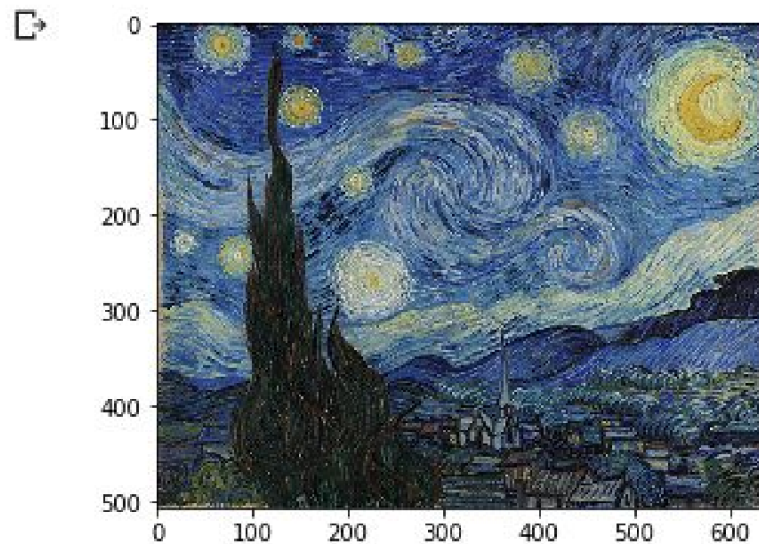
To run the model:

```
!python neural_style.py --content examples/2-content.jpg --styles examples/1-style.jpg --output out.jpg
```

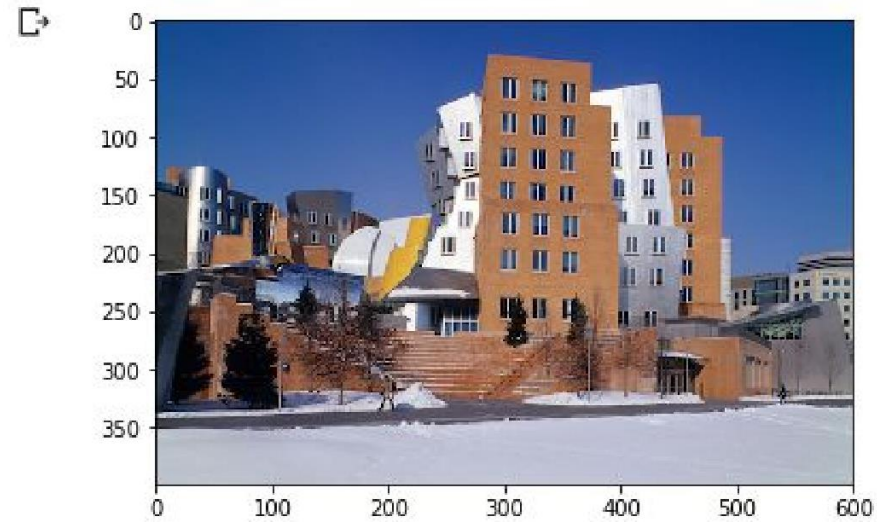
To visualize the inputs/outputs:

```
[10] %matplotlib inline  
  
import matplotlib.pyplot as plt  
import matplotlib.image as mpimg
```

```
img = mpimg.imread('examples/1-style.jpg')  
imgplot = plt.imshow(img)
```



```
[12] img = mpimg.imread('examples/2-content.jpg')  
imgplot = plt.imshow(img)
```



```
[13] img = mpimg.imread('out.jpg')  
imgplot = plt.imshow(img)
```

